

Developing Desktop 3D and VR applications:  
A detailed comparison between Godot, Unity and  
MAGES NXT

Bachelor / Master Thesis



**Author:** Anastasios Akoumianakis (Student ID: 4582)

**Supervisor:** Prof. Papagiannakis George

**Co-Supervisor:** Dr. Kamarianakis Manos

Department of Computer Science  
School of Sciences & Engineering  
University of Crete  
Heraklion, Greece

September 27, 2024

# Abstract

This thesis investigates and compares the differences between the Godot and Unity game engines through the development of a 3D escape room-style game in both environments. The research aims to provide insights into the strengths, weaknesses, and suitability of each engine for game development projects, particularly in the context of creating immersive 3D experiences. The methodology involves a comparative analysis of the features, performance, workflow, and flexibility of the two engines, supplemented by hands-on development experience. The findings of this study contribute to a better understanding of the capabilities and limitations of Godot and Unity, assisting game developers and designers in making informed decisions regarding engine selection for their projects.

Also, in the scope of this thesis, is the development of a Virtual Reality game using the MAGES NXT SDK. The specific framework offers unique features and capabilities that may appeal to certain developers. Through comparative analysis and practical experimentation, this research evaluates the suitability and potential advantages of the MAGES SDK for Desktop 3D or virtual reality 3D development projects.

# Declaration of Authorship

I, Anastasios Akoumianakis, declare that this thesis titled, "Developing Desktop 3D and VR applications:

A detailed comparison between Godot, Unity and MAGES NXT", and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

**Signature:**      Anastasios Akoumianakis

---

**Date:**              September 27, 2024

---

# Acknowledgements

I would like to express my gratitude to my supervisor Prof. Papagiannakis George for his continuous support and essential direction throughout the course of this project.

I would also like to thank Dr. Kamarianakis Manos, for their instructions and helpful advice.

# Contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>1</b>  |
| 1.1      | Modern Game Engines . . . . .                         | 1         |
| 1.1.1    | The Unity Game Engine . . . . .                       | 2         |
| 1.1.2    | The Godot Engine . . . . .                            | 2         |
| 1.1.3    | The MAGES NXT SDK . . . . .                           | 3         |
| 1.1.4    | Various other engines . . . . .                       | 3         |
| <b>2</b> | <b>Comparison specifications</b>                      | <b>5</b>  |
| 2.1      | Metrics used for the comparison . . . . .             | 5         |
| <b>3</b> | <b>Godot vs Unity</b>                                 | <b>7</b>  |
| 3.1      | Licensing . . . . .                                   | 7         |
| 3.2      | Programming Languages . . . . .                       | 7         |
| 3.3      | Platform Support . . . . .                            | 11        |
| 3.4      | Graphics Capabilities . . . . .                       | 12        |
| 3.5      | Asset Store . . . . .                                 | 15        |
| 3.6      | Community Support . . . . .                           | 17        |
| 3.7      | Ease of Use . . . . .                                 | 18        |
| 3.8      | Performance . . . . .                                 | 20        |
| 3.9      | Editor Customization . . . . .                        | 22        |
| 3.10     | VR Support . . . . .                                  | 25        |
| 3.11     | Built-in Features . . . . .                           | 29        |
| 3.12     | Code-Length Comparison: A Realistic Example . . . . . | 32        |
| 3.13     | Development Time . . . . .                            | 33        |
| 3.14     | Comparison Summary & Verdict . . . . .                | 34        |
| <b>4</b> | <b>Unity vs Mages SDK</b>                             | <b>37</b> |
| 4.1      | Introduction to the MAGES SDK . . . . .               | 37        |
| 4.2      | Key Features of Mages NXT . . . . .                   | 38        |
| 4.3      | Comparison . . . . .                                  | 44        |
| <b>5</b> | <b>Godot vs Mages NXT</b>                             | <b>45</b> |
| 5.1      | Introduction . . . . .                                | 45        |
| 5.2      | Out-of-the-Box Functionality . . . . .                | 45        |
| 5.3      | Documentation and Community Support . . . . .         | 45        |
| 5.4      | Coding Requirements . . . . .                         | 46        |
| 5.5      | Graphics Capabilities . . . . .                       | 46        |
| 5.6      | Multiplayer Support . . . . .                         | 47        |
| 5.7      | Comparison . . . . .                                  | 47        |

|          |                                         |           |
|----------|-----------------------------------------|-----------|
| <b>6</b> | <b>Comparison Summary &amp; Verdict</b> | <b>48</b> |
| 6.1      | Comparison Summary . . . . .            | 48        |
| 6.2      | Verdict . . . . .                       | 52        |
|          | <b>References</b>                       | <b>53</b> |
| <b>A</b> | <b>Appendix</b>                         | <b>55</b> |
| A.1      | Godot development impressions . . . . . | 55        |
| A.2      | Serializable Dictionary in C# . . . . . | 62        |

## List of Figures

|    |                                                                                      |    |
|----|--------------------------------------------------------------------------------------|----|
| 1  | Alien Isolation hosts one of the most advanced AI systems in gaming history. . . . . | 1  |
| 2  | Half Life Alyx liquid simulation. . . . .                                            | 1  |
| 3  | Showcase of 2D and 3D games. . . . .                                                 | 2  |
| 4  | Small GDScript file. . . . .                                                         | 8  |
| 5  | Godot's code editor. . . . .                                                         | 8  |
| 6  | Referencing objects from the scene in the code editor using drag-and-drop. . . . .   | 9  |
| 7  | The same Pressure Plate Script in C# (Top) and in GDScript (Bottom). . . . .         | 10 |
| 8  | Graphics Comparison between Unity (top) and Godot (bottom). .                        | 13 |
| 9  | Graphics Comparison between Unity (top) and Godot (bottom). .                        | 14 |
| 10 | Graphics Comparison between Unity (top) and Godot (bottom). .                        | 14 |
| 11 | Godot's Built-In Asset Library. . . . .                                              | 15 |
| 12 | Unity's Asset Store. . . . .                                                         | 16 |
| 13 | Unity's package manager. . . . .                                                     | 16 |
| 14 | Godot vs Unity benchmark performance. . . . .                                        | 21 |
| 15 | Godot's Editor. . . . .                                                              | 22 |
| 16 | Unity's editor. . . . .                                                              | 23 |
| 17 | Serializable Dictionary in-editor result. . . . .                                    | 25 |
| 18 | Unity's hand controllers. . . . .                                                    | 27 |
| 19 | Grabbing an object in Unity. . . . .                                                 | 27 |
| 20 | Godot's hand controllers. . . . .                                                    | 27 |
| 21 | Grabbing an object in Godot. . . . .                                                 | 28 |
| 22 | MAGES custom-made UI in Unity. . . . .                                               | 39 |
| 23 | Insert Action Begin. . . . .                                                         | 40 |
| 24 | Insert Action Completed. . . . .                                                     | 40 |
| 25 | Use Action Example. . . . .                                                          | 41 |
| 26 | Remove Action Begin. . . . .                                                         | 41 |
| 27 | Remove Action Complete. . . . .                                                      | 42 |
| 28 | Trajectory Action Example. . . . .                                                   | 42 |
| 29 | Activate Action Example. . . . .                                                     | 43 |
| 30 | Question Action Example. . . . .                                                     | 43 |
| 31 | Node List. . . . .                                                                   | 58 |
| 32 | Example Scene Hierarchy. . . . .                                                     | 59 |
| 33 | GDScript Abstract class. . . . .                                                     | 59 |
| 34 | Godot's key frame creation system. . . . .                                           | 60 |

|    |                                                    |    |
|----|----------------------------------------------------|----|
| 35 | Animation Player's UI . . . . .                    | 60 |
| 36 | Unity Serializable Dictionary . . . . .            | 62 |
| 37 | Serializable Dictionary in-editor result . . . . . | 63 |

## List of Tables

|   |                                    |    |
|---|------------------------------------|----|
| 1 | Summary Comparison Table . . . . . | 51 |
|---|------------------------------------|----|

# 1 Introduction

## 1.1 Modern Game Engines

A modern game engine is a robust software framework designed for the development and creation of video games. It provides a suite of tools and functionalities that enable developers to build, test, and deploy games efficiently. Modern game engines offer features such as real-time rendering, physics simulation (Figure 2), artificial intelligence (Figure 1, scripting, and asset management, which streamline the development process.



Figure 1: Alien Isolation hosts one of the most advanced AI systems in gaming history.

[2]



Figure 2: Half Life Alyx liquid simulation.

Modern game engines have evolved to support multiple platforms, including PCs, consoles, mobile devices, and virtual reality (VR) systems. They often come with integrated development environments (IDEs) that facilitate the design of both 2D and 3D games (Figure 3), enabling creators to focus on creativity and gameplay rather than technical details.



Figure 3: Showcase of 2D and 3D games.  
[14]

### 1.1.1 The Unity Game Engine

The Unity game engine [18], developed by Unity Technologies, is one of the most widely-used and versatile game engines in the industry, with a market share of 26.93% as of 2024 [1]. It supports a wide range of platforms including Windows, macOS, Linux, Android, iOS, and various gaming consoles. Unity is renowned for its ease of use, making it a popular choice for both beginners and experienced developers.

Unity is used in a variety of applications beyond gaming, including simulations, architectural visualizations, and interactive experiences, making it a versatile tool for many industries [18].

### 1.1.2 The Godot Engine

The Godot engine [10] is an open-source game engine under the MIT license, which is free to use and highly customizable. It is designed to be easy to use, with a focus on simplicity and productivity, particularly for 2D games. Moreover, Godot also offers robust tools for 3D game creation, such as a fully integrated 3D editor, a powerful physics engine, and support for advanced visual effects.

Godot’s community is active and growing, with a strong culture of open-source contributions, which means new features and improvements are constantly being added.

### 1.1.3 The MAGES NXT SDK

The MAGES NXT SDK [12] [13] is a state-of-the-art SDK built upon the Unity Engine, specifically designed for creating immersive XR (Extended Reality) simulations. Building on 20 years of research and evolving from the low-code MAGES 4.0 SDK to a no-code solution, it leverages cutting-edge technology to streamline the development process for high-quality medical simulations and other interactive experiences.

### 1.1.4 Various other engines

In addition to Unity and Godot, there are several other game engines that cater to different needs and preferences of developers:

- **Unreal Engine** Developed by Epic Games, Unreal Engine [8] is known for its high-fidelity graphics and powerful performance capabilities. It uses C++ and Blueprints (a visual scripting language) and is often used for AAA games and complex 3D projects.
- **CryEngine** Created by Crytek, CryEngine [6] is another high-performance engine known for its stunning visuals and advanced features. It is widely used for first-person shooters and other visually intensive games.
- **GameMaker Studio** Known for its ease of use in 2D game development, GameMaker Studio [11] uses a drag-and-drop interface along with its proprietary scripting language, GML. It is ideal for indie developers and hobbyists.
- **RPG Maker** Designed specifically for creating role-playing games, RPG Maker [7] offers a straightforward interface and pre-made assets, allowing developers to focus on storytelling and gameplay mechanics.

Each of these engines offers unique features and strengths, making them suitable for different types of projects and development teams. Selecting the right engine depends on factors such as the target platform, project scope, team expertise, and specific project requirements.

**The aim of the thesis** is to analyze and compare three major game development engines: Godot, Unity, and MAGES NXT. By examining the aforementioned key aspects, this thesis provides a detailed comparison of each engine's strengths and weaknesses. This analysis is intended to assist developers in selecting the most suitable engine for their projects and to contribute to the academic discourse on game development technologies through a clear and thorough evaluation of these widely-used tools.

The rest of this thesis is structured as follows: Starting in Section 2, the criteria used for comparing the game engines will be outlined, in order to ensure a clear and concise evaluation. In Section 3, a comprehensive comparison of Godot and Unity is presented.

Unity is also compared with MAGES SDK in Section 4, highlighting the unique features and functionalities of each.

Lastly, the differences between Godot and MAGES NXT are described in Section 5, focusing on the advantages and disadvantages of each development tool.

The thesis concludes with a summary of findings, a comparison table for quick reference and a verdict in Section 6.

In Appendix A, additional content is provided of the author's first development impressions of Godot and Unity, offering insights that extend somewhat beyond the primary scope of this thesis.

## 2 Comparison specifications

Comparison specifications are imperative for a comparison of game engines because they provide a clear, concise, and organized way to highlight the differences and similarities between the game engines. Our goal is to facilitate an informed decision-making process for developers by allowing readers to quickly grasp the key features, capabilities, and limitations of each engine.

### 2.1 Metrics used for the comparison

In the comparison of different game engines, several key metrics are used to evaluate their features and capabilities. Here is a brief description of each metric used in the comparison:

- **Licensing.** Licensing refers to the terms under which the game engine can be used. It includes information about costs, usage rights, and any royalties that might be owed. Licensing terms can greatly impact the accessibility and financial feasibility of using an engine for different types of projects.
- **Programming Languages.** This metric describes the programming languages supported by the game engine. The choice of programming language affects the ease of learning, performance, and flexibility in game development. Common languages include C#, C++, and scripting languages like GDScript.
- **Platform Support.** Platform support indicates the range of operating systems and devices that the game engine can target. Broad platform support allows developers to reach a wider audience by deploying their games on multiple platforms such as Windows, macOS, Linux, Android, iOS, and gaming consoles.
- **Graphical Capabilities.** Graphical capabilities refer to the engine's ability to render 2D and 3D graphics. This includes support for advanced rendering techniques, shaders, lighting, and other visual effects that contribute to the overall aesthetic and performance of the game.
- **Asset Store.** An asset store is a marketplace where developers can purchase or download free assets like 3D models, textures, sounds, and scripts. A rich asset store can significantly speed up development by providing ready-made resources that can be easily integrated into projects.
- **Community Support.** Community support refers to the active user base and availability of resources such as forums, answers to common questions,

and documentation. Strong community support can be invaluable for troubleshooting, learning, and sharing knowledge.

- **Ease of use.** Ease of use measures how user-friendly the game engine is, particularly for beginners. This includes the intuitiveness of the interface, the availability of tutorials, and the overall learning curve associated with the engine.
- **Performance.** Performance assesses how well the engine can handle demanding tasks, such as rendering complex scenes and running heavy computations. High performance is crucial for creating smooth and responsive games, especially those with high graphical fidelity.
- **Editor Customization.** Editor customization examines how much the development environment can be tailored to suit the developer's workflow. This includes the ability to create custom tools, modify the interface, and script editor behaviors.
- **VR Support.** VR support indicates the engine's capabilities for developing virtual reality applications. This includes built-in tools and optimizations for VR development, as well as compatibility with VR hardware.
- **Built-in Features.** Built-in features refer to the tools and functionalities that come pre-packaged with the engine. These can include visual editors, animation tools, physics engines, and other utilities that aid in game development.
- **Development Time.** Development time refers to the duration required to create a game using the engine. This metric considers the efficiency and productivity of the engine's tools and workflows, impacting the overall time to market for a game project.

## 3 Godot vs Unity

### 3.1 Licensing

#### Godot

Godot is completely free and open-source, distributed under the MIT License. This license grants developers full access to the source code, allowing them to modify and redistribute it without any restrictions. This open-source nature fosters a collaborative environment where the community can contribute to the engine's development. There are no licensing fees or revenue caps, making Godot an attractive option for both hobbyists and professional developers.

#### Unity

Unity offers a free personal version, which is available to developers with revenue under a certain threshold. For larger projects and commercial use, Unity provides professional versions that come with a fee, unlocking advanced features and services. These professional versions include access to Unity's cloud services, advanced performance analytics, and priority support. While the personal version is sufficient for many small projects, larger studios and enterprises typically opt for the paid versions to take advantage of the additional features and support.

#### Comparison

Both engines provide accessible options for developers at different stages. Godot's completely free and open-source model offers unmatched flexibility, while Unity's tiered licensing provides scalability for commercial projects.

### 3.2 Programming Languages

#### Godot and GDScript

Godot primarily uses GDScript ([https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/gdscript\\_basics.html](https://docs.godotengine.org/en/stable/tutorials/scripting/gdscript/gdscript_basics.html)), a language specifically designed for the engine, though it also supports C#, VisualScript, and C++. For the purposes of this thesis, the focus will be on GDScript due to its deep integration with Godot and its recommendation by the engine's developers.

```

extends Area3D

@onready var rootNode: Node3D = $".."
@onready var movableItem: Interactable = $"../MovableItem"
@onready var objectId: int = rootNode.objectId
var thisId = 0

func _process(delta):
    if overlaps_body(movableItem):
        movableItem.transform = transform

```

Figure 4: Small GDScript file.

GDScript is a dynamically-typed language with syntax resembling Python, as shown in Figure 4. It offers features common to modern programming languages, making it easy to learn and use. This simplicity allows developers to concentrate on implementing features rather than grappling with a new language. Additionally, its straightforward syntax significantly reduces code length.

One of GDScript's main advantages is its seamless integration with Godot's editor. This integration provides all the features expected of modern IDEs such as code editing, highlighting, auto-completion and real-time error messages, all in the same space as the scene editor which can be seen in Figure 5. Due to the scene editor being next to the code editor, the code editor provides various quality-of-life features, such as quick object referencing from the scene hierarchy and asset folders (Figure 6), eliminating the need for an external text editor.

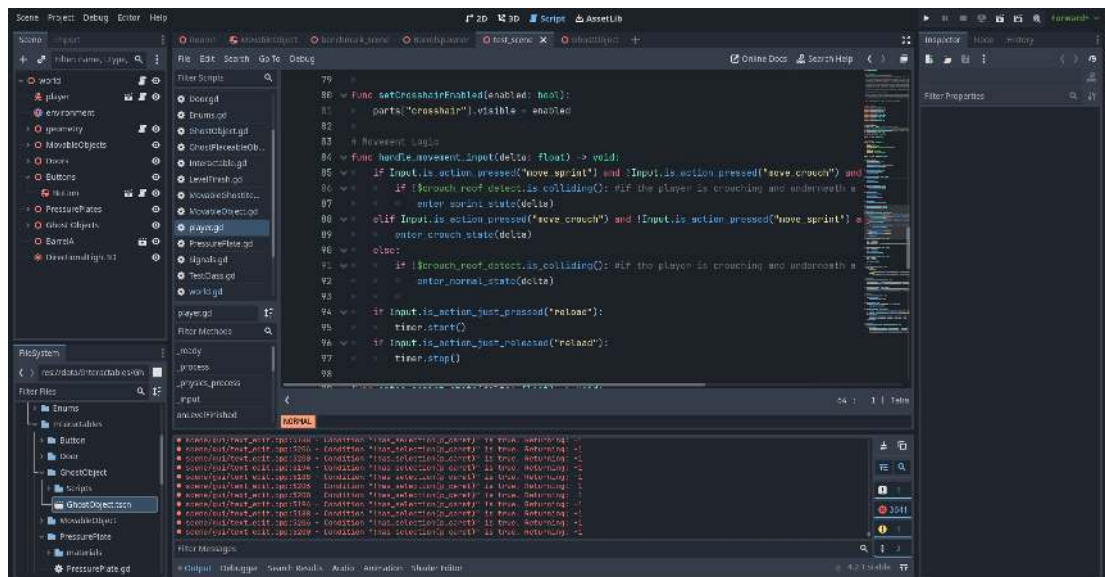


Figure 5: Godot's code editor.



Figure 6: Referencing objects from the scene in the code editor using drag-and-drop.

However, GDScript has its drawbacks. Godot is still under active development, and GDScript has undergone many changes that significantly impact development practices. Established best practices for GDScript are still forming, leading to a lack of tutorials and forum support that are crucial for developers.

The documentation for GDScript is also fragmented and often inadequate. Although Godot's current version is 4.0, much of the documentation still pertains to deprecated practices from version 3.0, making it challenging for developers to find accurate and up-to-date information. This documentation gap means that developers may struggle to find help when encountering issues.

## Unity and C#

In contrast, Unity uses C#, a statically-typed language widely employed for .NET applications and game development. C# has a more complex syntax than GDScript, resulting in lengthier code.

The verbosity and strictness of C# can enhance code readability and maintainability. As a widely used modern language, C# offers numerous features that developers might need, and, crucially, its features and documentation are stable. Best practices for C# are well-established, and there is an abundance of tutorials and learning materials available, both free and paid.

While C# may be more difficult to learn and require more lines of code, developers have extensive resources at their disposal to master the language. These resources include free online tools and comprehensive paid courses for those seeking an in-depth understanding.

## Comparison

To supply a better understanding of the comparison, I will include the complete script for the "pressure plate" mechanic as developed in both engines, using C# for Unity and GDScript for Godot in Figure 7.

```

3  public class PressurePlate : SignalSender {
4      [SerializeField] private SerializableDictionary<GameObject, ObjectInteractStates> onStoppedBeingPressed;
5      [SerializeField] private Material pressurePlateOffMaterial;
6      [SerializeField] private Material pressurePlateOnMaterial;

7
8      private int numberOfCollisions = 0;
9
10     private void Start() {
11         var startingMaterial = numberOfCollisions > 0 ? pressurePlateOnMaterial : pressurePlateOffMaterial;
12         GetComponent<MeshRenderer>().material = startingMaterial;
13     }
14
15     private void OnTriggerEnter(Collider other) {
16         if (numberOfCollisions <= 0) {
17             SendSignals();
18             GetComponent<MeshRenderer>().material = pressurePlateOnMaterial;
19         }
20         numberOfCollisions++;
21     }
22
23     private void OnTriggerExit(Collider other) {
24         numberOfCollisions--;
25         if (numberOfCollisions <= 0) {
26             SendSignals(onStoppedBeingPressed);
27             GetComponent<MeshRenderer>().material = pressurePlateOffMaterial;
28         }
29     }
30 }

```

```

1  extends Area3D
2
3  @export var affectedObjectIds: Array[int]
4  @export var OnPressurePlatePressed: Array[Enums.ObjectInteractBehavior]
5  @export var OnPressurePlateReleased: Array[Enums.ObjectInteractBehavior]
6
7  @onready var mesh: MeshInstance3D = $MeshInstance3D
8  var plateActiveMaterial: Material = preload("res://data/Interactables/PressurePlate/materials/
9  var plateNotActiveMaterial: Material = preload("res://data/Interactables/PressurePlate/materials/
10 var isPressed := false
11
12 func _process(delta):
13     setIsPressed(has_overlapping_bodies())
14     pass
15
16 func setIsPressed(newValue: bool):
17     if newValue == isPressed: return
18     isPressed = newValue
19
20     if isPressed:
21         Signals.pressurePlatePressed.emit(affectedObjectIds, OnPressurePlatePressed)
22         mesh.set_surface_override_material(0, plateActiveMaterial)
23     else:
24         Signals.pressurePlateReleased.emit(affectedObjectIds, OnPressurePlateReleased)
25         mesh.set_surface_override_material(0, plateNotActiveMaterial)

```

Figure 7: The same Pressure Plate Script in C# (Top) and in GDScript (Bottom).

In conclusion, while GDScript offers many advantages, including ease of learning and deep integration with Godot, it is hindered by its evolving nature and fragmented documentation. Conversely, C# provides stability, extensive resources, and well-established best practices, making it currently superior in the programming language department for game development. In the future, GDScript's documentation and community support may improve, but for now, C# holds the edge.

### 3.3 Platform Support

#### Godot

Godot is known for its broad platform support, allowing developers to export games to various systems with minimal hassle. The supported platforms include:

- **Desktop:** Windows, macOS, Linux
- **Mobile:** Android, iOS
- **Web:** HTML5
- **Console:** PlayStation, Xbox, Nintendo Switch (though console support often requires third-party services)

Godot's export process is streamlined, making it relatively straightforward to deploy games across these platforms. However, it's important to note that while Godot offers official support for many platforms, some (particularly consoles) may require additional steps or reliance on third-party solutions to achieve optimal performance and integration.

#### Unity

Unity excels in platform support, offering extensive and mature capabilities for exporting games to a wide array of platforms. The supported platforms include:

- **Desktop:** Windows, macOS, Linux
- **Mobile:** Android, iOS
- **Web:** HTML5, WebGL
- **Console:** PlayStation, Xbox, Nintendo Switch
- **Others:** AR/VR devices, smart TVs, and more

Unity’s platform support is one of its strongest features, providing developers with robust tools and documentation to ensure smooth deployment across various systems. The integration for console platforms is particularly well-developed, often not requiring third-party services and offering a more direct and streamlined workflow.

## **Comparison**

While both Godot and Unity offer extensive platform support, Unity’s broader range of supported platforms, ease of deployment, and superior quality of integration, particularly for consoles and emerging technologies, make it the clear winner in this category.

## **3.4 Graphics Capabilities**

### **Godot**

Godot’s graphics capabilities are functional but limited in their fidelity. Each Godot scene offers basic graphical elements such as simple shapes, minimal lighting and shadows. While these features are sufficient for simple projects or prototyping, they fall short when developing high-fidelity visuals.

Godot’s material and shader system, while flexible, requires more effort from developers to achieve advanced visual effects. Developers often need to create custom shaders to obtain desired results, which can be time-consuming and complex. The default lighting in Godot is basic, and achieving realistic lighting effects often necessitates significant manual adjustments and expertise.

While Godot is continually improving and adding new graphical features, the current offerings are more suited for 2D games, indie projects, or developers willing to invest extra time in fine-tuning their graphics.

### **Unity**

Unity excels in graphics capabilities, offering a robust set of tools and features out of the box. Each Unity project starts with access to high-quality materials, advanced shaders, dynamic lighting, and realistic shadows. This foundation allows developers to create visually stunning games without needing to build these features from scratch.

Unity’s extensive library of pre-built assets, combined with its powerful rendering pipelines, provides a significant advantage in developing graphically rich environments. The High Definition Render Pipeline (HDRP) and Universal Render Pipeline (URP) enable developers to achieve high-quality visuals tailored to

various performance requirements.

Additionally, Unity’s graphical capabilities are supported by comprehensive documentation and community resources, making it easier for developers to implement advanced visual effects and troubleshoot issues.

### Comparison

When comparing the graphics capabilities of Godot and Unity, the differences are stark. Unity’s robust set of built-in tools, high-quality default materials, and advanced lighting and shadow systems give it a significant edge over Godot and there are studies that support all of the aforementioned arguments, see [16].

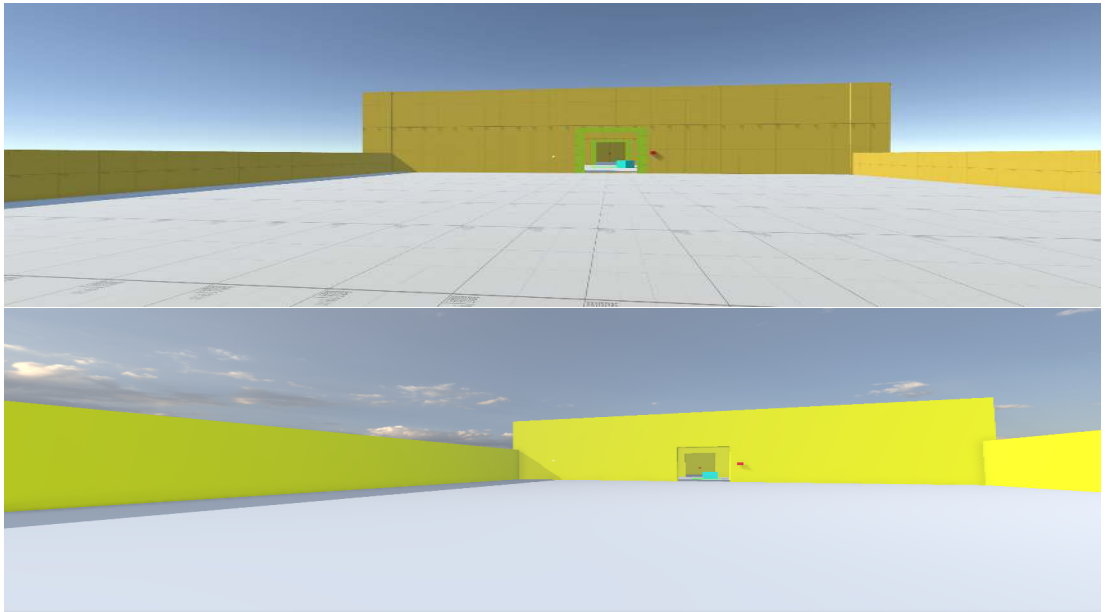


Figure 8: Graphics Comparison between Unity (top) and Godot (bottom).

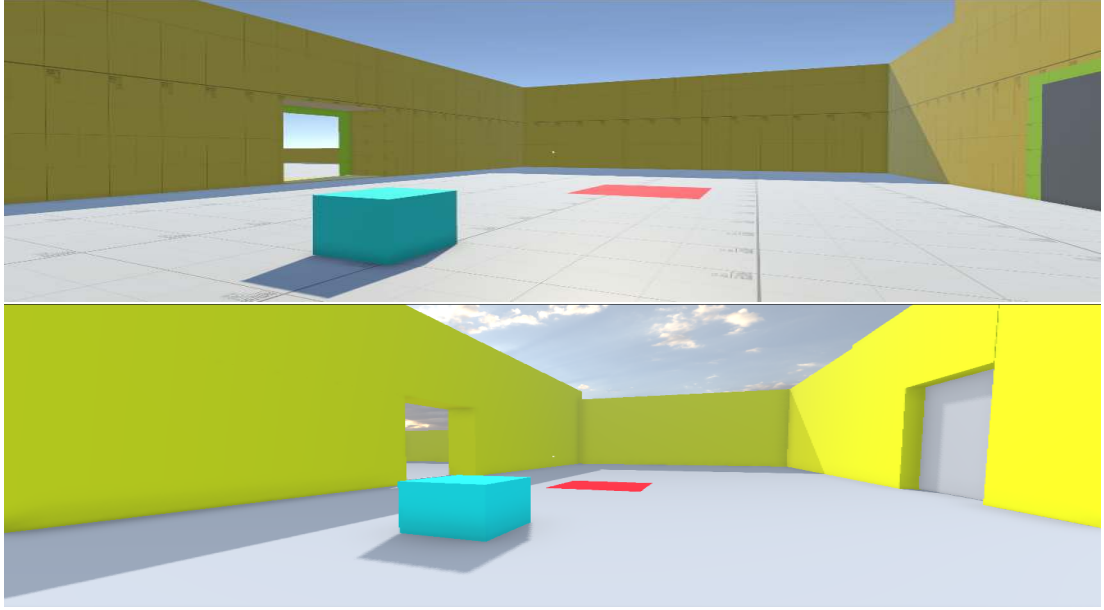


Figure 9: Graphics Comparison between Unity (top) and Godot (bottom).

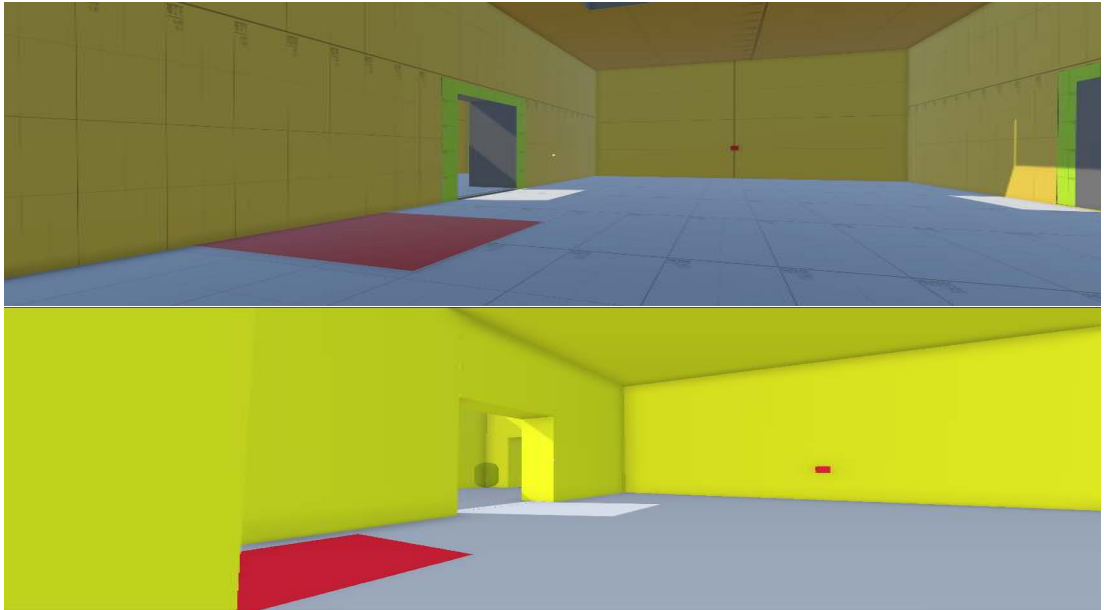


Figure 10: Graphics Comparison between Unity (top) and Godot (bottom).

As shown in the figures 8, 9, 10, Unity's default objects appear textured while Godot's do not. In Unity, adding the textured image on the objects required no extra effort and was as simple as choosing a lit material from Unity's default selection. In Godot to achieve the same result as Unity's would necessitate additional effort, as the process is not as straightforward. This highlights a key difference in the usability and efficiency of these engines in handling textures and materials.

In conclusion, Unity is the clear winner in the graphics capabilities category. Its superior default assets, advanced lighting and shading systems, and comprehensive support resources make it the preferred choice for developers aiming to create visually impressive games.

### 3.5 Asset Store

#### Godot

Godot features a built-in asset library (Figure 11) where users can create, share, and download resources. This asset library is entirely free, providing a range of tools, scripts, and graphical assets that can help developers accelerate their projects. The free nature of the Godot asset library makes it accessible to all users, regardless of budget [9].

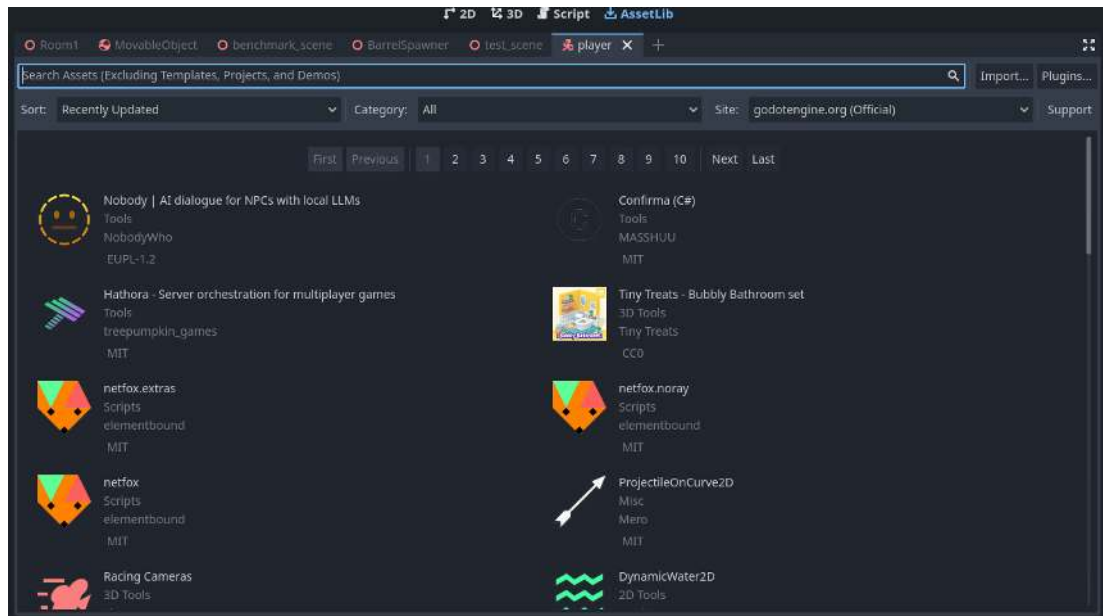


Figure 11: Godot’s Built-In Asset Library.

#### Unity

Unity boasts a comprehensive asset store (Figure 12) that offers both free and paid assets. The range of available assets is vast, covering everything from simple scripts and tools to complex graphical assets and complete project templates. This diversity is a direct result of Unity’s larger and more active community, which continuously contributes to the asset store [17].

Many of the free assets and packages available on Unity’s asset store have become industry staples, significantly impacting game development by providing high-quality resources that developers can integrate into their projects. The

availability of paid assets also ensures that developers have access to premium resources, often created by professional developers and artists.

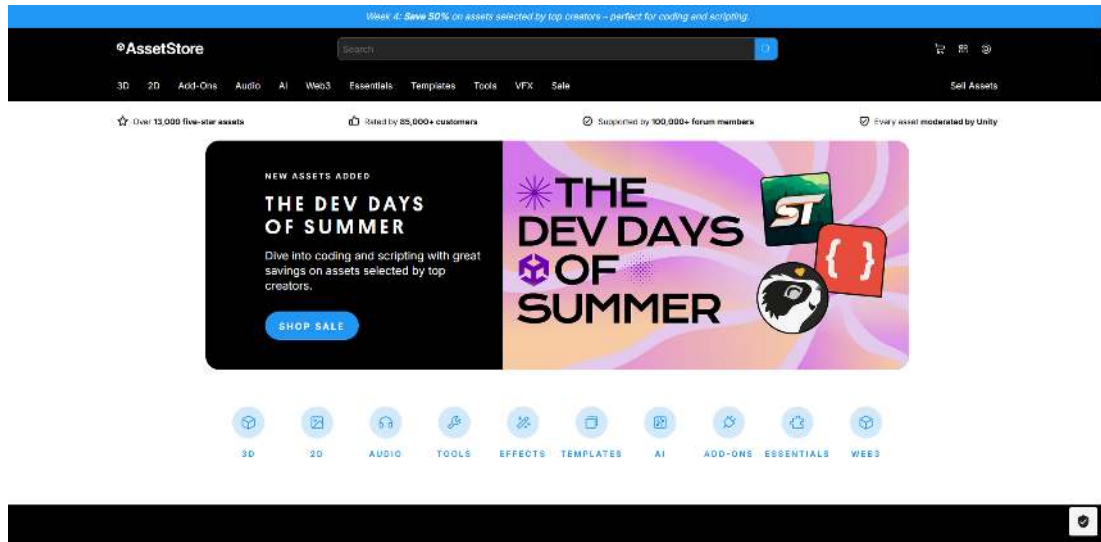


Figure 12: Unity’s Asset Store.

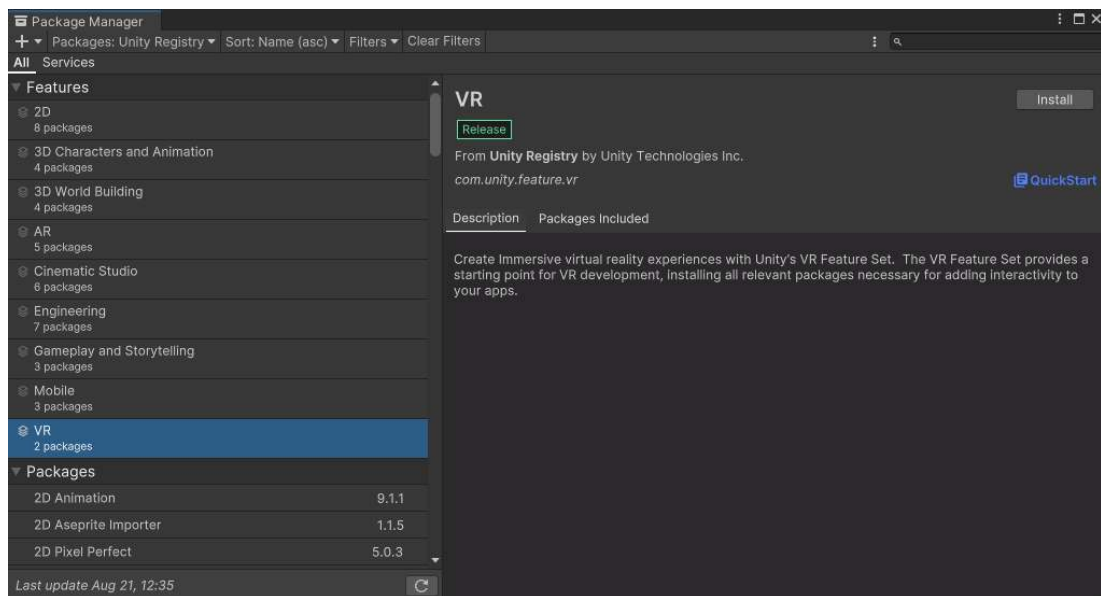


Figure 13: Unity’s package manager.

## Comparison

The quantity and quality of assets available in Godot’s library are limited compared to Unity. The community support for Godot, while growing, is not as extensive as Unity’s, resulting in fewer assets and less frequent updates. While there are some high-quality assets available, the overall selection is narrower, which can constrain developers looking for specific tools or high-fidelity graphical elements.

Godot has also integrated their asset store seamlessly into the editor while Unity's asset store is solely available through a browser in order to increase the editor's performance. Despite the asset store not being available in the editor, downloading and integrating assets is streamlined and efficient through the Unity Editor's package manager (Figure 13). However, Unity's asset store is generally regarded as more robust due to the sheer volume and quality of assets available.

Unity is the clear winner in the asset store category. Its extensive selection of both free and paid assets, supported by a large and active community, provides developers with a wealth of resources to enhance their projects. The higher quality and impact of Unity's assets make it the preferred choice for developers seeking comprehensive and readily available resources.

### 3.6 Community Support

#### Godot

Godot's community plays a vital role in its development, especially considering that it is an open-source engine. This means that anyone can contribute to the engine's improvement by reporting bugs, suggesting features, or even directly modifying the engine's codebase. The open-source nature fosters a collaborative environment where developers can have a significant impact on the engine's evolution.

The Godot community is enthusiastic and growing, providing support through forums, Discord channels, and other online platforms.

#### Unity

Unity benefits from a massive and active community that significantly enhances its development ecosystem. The large user base contributes a wealth of tutorials, forums, and online courses, making it easier for new developers to learn the engine and for experienced developers to find solutions to specific problems. Unity's extensive documentation is regularly updated and provides detailed information on all aspects of the engine, ensuring developers have access to the latest and most accurate resources.

The Unity Asset Store is a direct reflection of this strong community support, offering a vast array of assets, both free and paid, that developers can use to enhance their projects. Many of these assets have become industry staples, significantly influencing game development practices.

Unity's community also drives the creation of third-party tools and extensions, further expanding the engine's capabilities and providing developers with even more options to streamline their workflow and improve their projects.

## Comparison

Godot's community is very supportive and collaborative, but it still is relatively small compared to Unity's. This smaller community size means fewer tutorials, resources, and third-party tools are available. The documentation for Godot, while improving, is still somewhat fragmented and often lags behind the latest engine updates, making it challenging for developers to find current and comprehensive information.

Unity wins in the community support category. Its large, active community provides extensive resources, robust documentation, and a wealth of third-party tools and assets that significantly benefit developers. While Godot's open-source nature allows for impactful community contributions and its community is growing and enthusiastic, Unity's established and comprehensive support system gives it the clear edge in this area.

## 3.7 Ease of Use

### Godot

Godot is renowned for its all-in-one interface, where the 2D and 3D scene editor, script editor, and asset store coexist seamlessly. This integrated environment provides a simple and intuitive UI that facilitates a smooth workflow for developers. The GDScript language, resembling Python, is straightforward to learn, allowing developers to start writing scripts quickly and efficiently.

Godot's strength lies in its out-of-the-box functionality. The engine comes with a rich set of pre-built tools and components that significantly ease the development process. Some of the key tools include:

- **CharacterBody:** Simplifies character movement and physics.
- **VehicleBody and VehicleWheels:** Provides built-in support for vehicle physics and controls.
- **Timer:** Enables precise timing and scheduling of events.
- **Raycast:** Facilitates raycasting operations for collision detection and interaction.
- **Canvas/UI items:** Offers a comprehensive set of tools for creating user interfaces.
- **AudioStreamPlayer:** Manages audio playback with ease.

- **BoneAttachment:** Allows objects to be attached to specific bones in a skeleton.
- **Object Joints:** Provides various types of joints for creating complex mechanical systems.
- **Path and PathFollow (Splines):** Supports path-based movement and animations.
- **Skeleton (Bone Hierarchy):** Manages skeletal structures for animations.
- **SpringArm:** Automatically adjusts objects' positions to avoid collisions.

These tools make Godot feel like a set of "building blocks" where developers can intuitively piece together various components to build their games. This approach significantly reduces the need to develop basic functionality from scratch, making the development process more efficient and user-friendly.

Appendix A includes a more in-depth look into Godot's ease of use and development style in support of the aforementioned arguments.

## Unity

Unity also offers a clean and intuitive UI, but it lacks some of the basic functionality provided by Godot out of the box. While Unity's interface is user-friendly, developers often need to create many foundational components themselves or rely on external assets and plugins to achieve similar functionality.

For instance, features like character movement, vehicle physics, timers, ray-casting, and UI elements typically require additional scripting or third-party assets in Unity. This need to develop or integrate these elements from scratch can increase the complexity and time required for development, especially for beginners or small teams.

However, Unity's extensive asset store and community resources somewhat mitigate this challenge by offering a plethora of free and paid assets that can help fill these gaps. Still, the initial setup and configuration can be more demanding compared to Godot's more immediately usable tools.

## Comparison

Godot is the winner in the ease of use category. Its all-in-one interface and rich set of out-of-the-box tools make it exceptionally intuitive and user-friendly, allowing developers to focus more on creativity and less on configuration. While Unity offers a polished and user-friendly interface, the need to develop basic functionality from scratch can make the initial development process more cumbersome.

### 3.8 Performance

#### Godot

Godot, while impressive for its ease of use and integrated tools, struggles with performance in more complex scenes. Some of the notable performance issues include:

- **Lower Quality Assets and Rendering:** Godot's rendering engine is not as advanced as Unity's, leading to lower quality assets and visual effects. Complex scenes with detailed models and textures can result in significant performance hits.
- **Subpar Shadows and Lighting:** The shadow quality in Godot often leaves much to be desired. The lighting system, while functional, lacks the sophistication found in Unity, which can impact the visual fidelity and performance of games.
- **Physics Optimization:** Godot's physics engine is less optimized compared to Unity's. Physics calculations and interactions, especially in 3D environments, can lead to performance bottlenecks. This lack of optimization becomes apparent in scenes with numerous physics objects and interactions.

#### Supporting Data

Several sources and user reports consistently highlight these performance disparities:

- **Rendering Performance:** Unity's use of HDRP and URP offers significantly better rendering performance and visual quality compared to Godot's built-in renderer [16].
- **Physics Engine:** Benchmarks and user feedback often cite Unity's PhysX-based engine as more robust and efficient, particularly for complex simulations and interactions [16], [4].
- **Asset Handling:** Unity's ability to manage high-quality assets with minimal performance impact is a well-documented advantage, making it the preferred choice for developers aiming for high-end graphics and complex scenes [16].

For this specific case, the same benchmark scene was created in both engines and measured the frame-rate performance in terms of frames-per-second which can be observed in Figure 14.

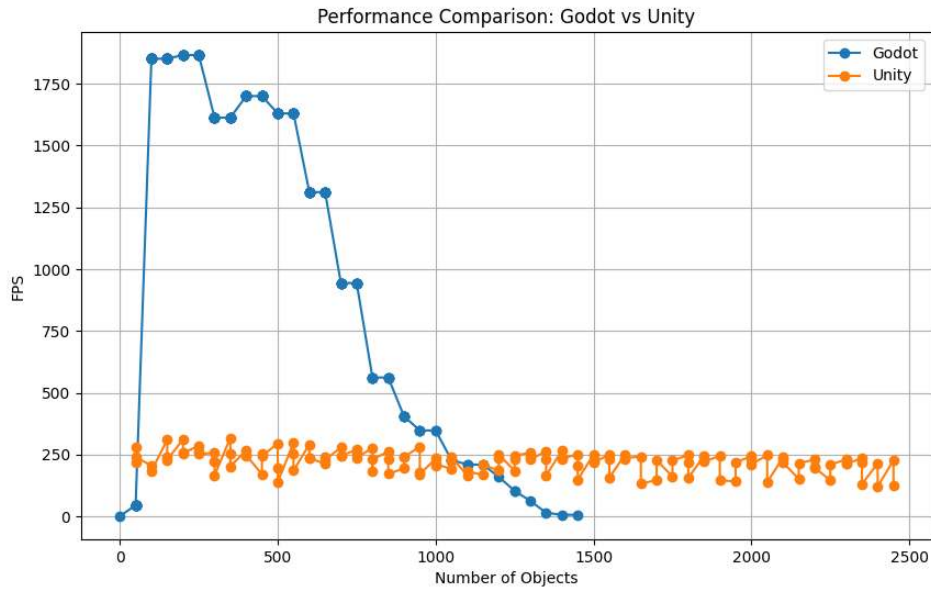


Figure 14: Godot vs Unity benchmark performance.

It can be observed that while Godot’s uncapped framerate is higher in the beginning, it starts to wane rapidly with the increasing number of objects in the scene, even reaching the range of 0-10 frames per second, while Unity’s remains steady throughout the whole benchmark, well after Godot’s game has become unplayable.

## Unity

Unity is well-known for its high performance, particularly in 3D rendering. The engine is optimized to handle complex scenes with high-quality assets efficiently, making it a popular choice for developers aiming to create visually stunning and resource-intensive games. Key aspects of Unity’s performance include:

- Advanced Rendering Pipelines:** Unity offers both the High Definition Render Pipeline (HDRP) for high-fidelity graphics and the Universal Render Pipeline (URP) for more scalable performance across different devices. These pipelines allow for advanced lighting, shadows, and post-processing effects while maintaining performance [15].
- Efficient Asset Handling:** Unity is capable of handling high-resolution textures, detailed models, and complex shaders without significant performance degradation. This capability is crucial for AAA games and high-end projects.

- **Optimized Physics Engine:** Unity’s physics engine, based on NVIDIA’s PhysX, is highly optimized for both 2D and 3D physics calculations, ensuring smooth performance even in physics-heavy scenes [4].
- **Extensive Performance Tools:** Unity provides robust profiling and performance analysis tools that help developers optimize their games effectively.

## Comparison

In conclusion, Unity is the clear winner in the performance category. Its advanced rendering pipelines, efficient asset handling, and optimized physics engine make it the superior choice for developers aiming to create high-performance, visually stunning games. Godot, while powerful in its own right, struggles to match Unity’s performance, particularly in more demanding 3D environments and complex scenes.

## 3.9 Editor Customization

### Godot

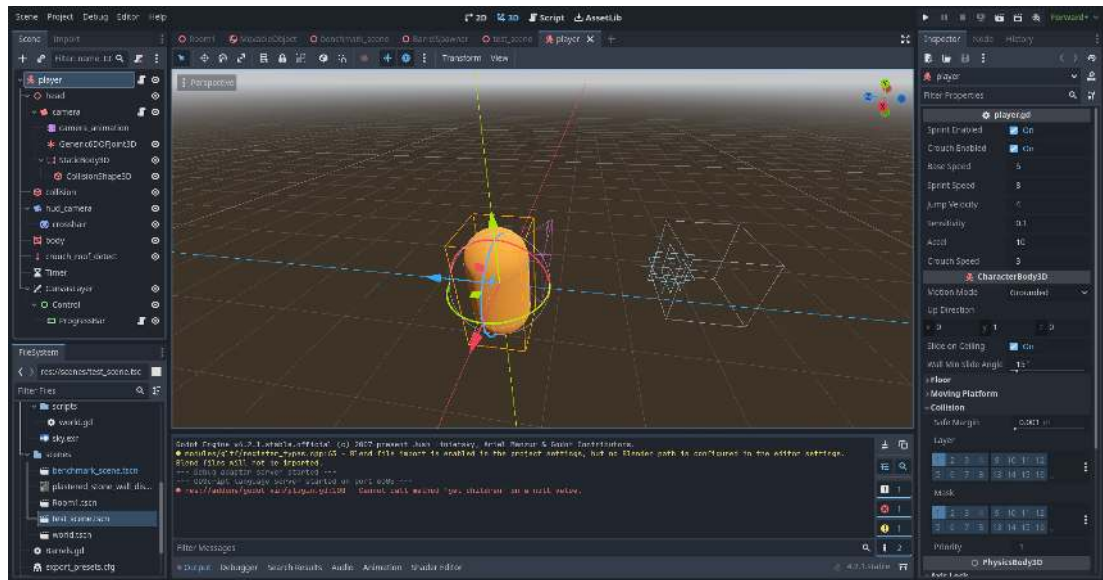


Figure 15: Godot’s Editor.

Godot offers a high degree of editor customization [3] (Figure 15), which is one of its standout features. The engine is designed to be flexible and adaptable to various workflows, making it a powerful tool for developers who prefer a tailored development environment. Key aspects of Godot’s editor customization include:

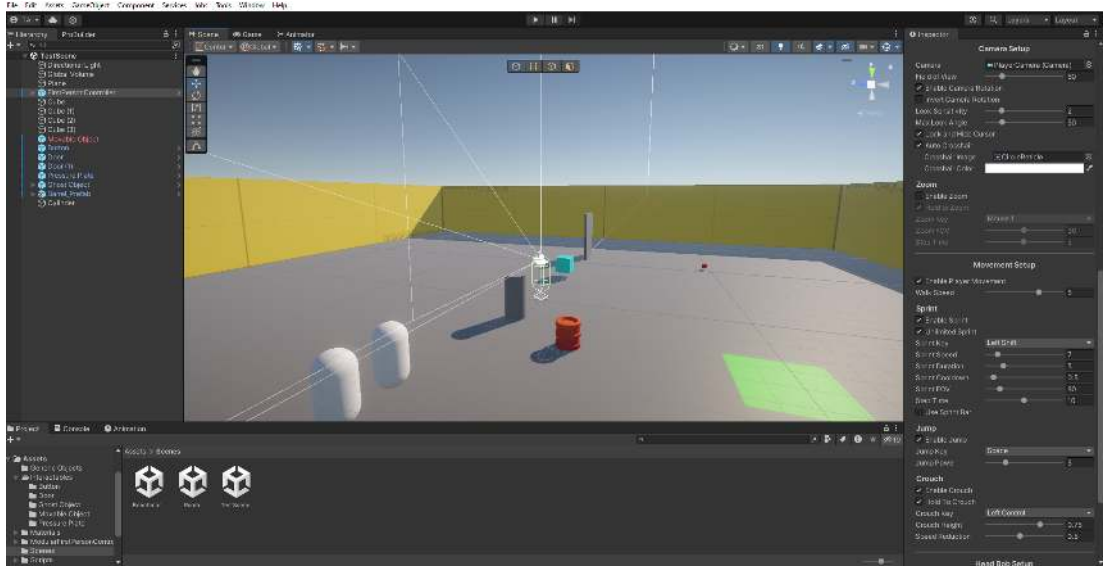


Figure 16: Unity's editor.

- **Scriptable Editor:** Godot allows developers to extend and customize the editor using its scripting language, GDScript, as well as C# and VisualScript. This enables the creation of custom tools and plugins directly within the editor.
- **Docking System:** The editor's interface is highly modular, featuring a docking system that allows developers to rearrange panels and windows according to their preferences. This flexibility helps in creating a workspace that suits individual development needs.
- **Custom Editor Plugins:** Developers can create custom plugins to add new functionality or modify existing features within the editor. These plugins can be shared and reused across different projects, enhancing productivity.
- **Inspector Customization:** The inspector in Godot can be extended to include custom properties and editors for specific node types, enabling more intuitive and efficient property management.
- **Theme Customization:** Godot supports theme customization, allowing developers to change the visual appearance of the editor to match their aesthetic preferences or improve readability and comfort.

## Unity

Unity also excels in editor customization[5], offering robust features that allow developers to tailor the environment to their needs (Figure 16). Unity's extensive

customization options are one of the reasons it is favored by many professional developers. Key features of Unity’s editor customization include:

- **Extensible Editor:** Unity’s editor is highly extensible through the use of C#. Developers can create custom editor windows, inspectors, and tools, which can be integrated seamlessly into the Unity environment.
- **Custom Editor Windows:** Unity allows the creation of custom editor windows, enabling developers to design specialized tools and interfaces that can simplify complex tasks or streamline workflows.
- **Property Drawers and Custom Inspectors:** Unity supports property drawers and custom inspectors, which provide enhanced control over how serialized fields are displayed and interacted with in the editor. This is particularly useful for complex data structures and custom scripts.
- **Editor Scripting API:** Unity’s extensive editor scripting API offers powerful capabilities for creating custom tools and modifying the editor’s behavior, making it possible to automate repetitive tasks and improve productivity.
- **Asset Store Extensions:** Unity’s Asset Store includes numerous editor extensions that can be purchased or downloaded for free, further expanding the customization possibilities.

### Author’s Experience

Based on the author’s experience, developing a 3D escape room game with Unity, the deep integration with Unity’s libraries proved invaluable. Thanks to C#, the author was able to write serializable dynamic dictionaries that integrated seamlessly with Unity’s scene editor (Appendix A, section A.2). This customization enabled faster development and more efficient management of game data directly within the editor as can be seen in Figure 17. The ability to create and manipulate complex data structures using custom tools in the editor significantly streamlined the development process and enhanced productivity.



Figure 17: Serializable Dictionary in-editor result.

## Comparison

While both Godot and Unity offer robust editor customization features, Unity has the edge due to its extensive scripting capabilities with C#, integration with a vast library of extensions from the Asset Store, and a highly extensible editor environment. Unity's customization options, combined with a large and active community, make it the preferred choice for developers looking to create a highly personalized and efficient development workflow.

### 3.10 VR Support

#### Unity

Unity offers extensive tools to make VR development accessible and efficient. The engine is equipped with a wide range of libraries and templates that provide out-of-the-box VR functionality, making it easier for developers to dive into VR projects. Key aspects of Unity's VR support include:

- **Comprehensive VR Libraries:** Unity provides a robust set of VR libraries that simplify the process of integrating VR features into games. These libraries handle various aspects of VR development, such as head tracking, hand interactions, and spatial audio.
- **VR Templates:** Unity Hub offers VR templates that showcase Unity's VR capabilities. These templates provide a solid foundation for new projects, allowing developers to start with a functional VR setup and modify it to suit their needs.
- **VR Simulation Tools:** One of Unity's standout features is its ability to simulate VR headsets without requiring physical hardware. Developers can easily simulate VR environments by clicking and dropping components,

making it possible to test and develop VR applications even without a VR headset. These simulation tools work seamlessly and help streamline the development process.

- **Thorough Documentation:** Unity’s XR Tools have been meticulously documented, providing developers with a comprehensive and detailed reference for every aspect of the platform. In addition to this extensive documentation, there is a wealth of tutorials and active forums available, offering invaluable guidance and support.

### Author’s Experience

In this thesis, the task was to develop VR games without having access to a VR headset. Unity’s VR simulation tools proved invaluable in this context. The ability to simulate VR headsets and interactions directly within the Unity editor allowed for effective development and testing of VR functionalities, bypassing the need for physical VR hardware. This feature significantly accelerated the development process and made it feasible to create VR experiences under hardware constraints. When a VR headset was eventually acquired, the process continued seamlessly, requiring only the connection of the headset to begin testing.

### Key differences of Godot vs Unity’s VR Support

The most notable difference between the VR support of the two engines is the movement options given to the playable character.

Unity’s tools provide a complete XR rig for player control, featuring controller models that serve as virtual hands, see Figure 18. These controllers offer a full suite of VR application functions, including camera movement and rotation, teleportation, and object interaction such as grabbing and manipulating items (see Figure 19).

Godot’s tools are dispersed throughout the library, requiring the developer to manually combine components to create a playable character. These tools support functions such as moving, rotating the camera, teleporting, picking up objects, and interacting with items. Instead of providing fully functional controllers like Unity, Godot offers hand models that developers must configure individually, see Figure 20. Each desired function must be manually added to the corresponding hand model, and due to the nature of Godot’s hands, each object also needs a specific grab hand pose for when it is picked up (see Figure 21). This, coupled with inadequate documentation, makes developing the player controller for Godot a tedious and challenging task. Developers often struggle to fully understand

the available tools, and those they do find lack straightforward implementation methods.

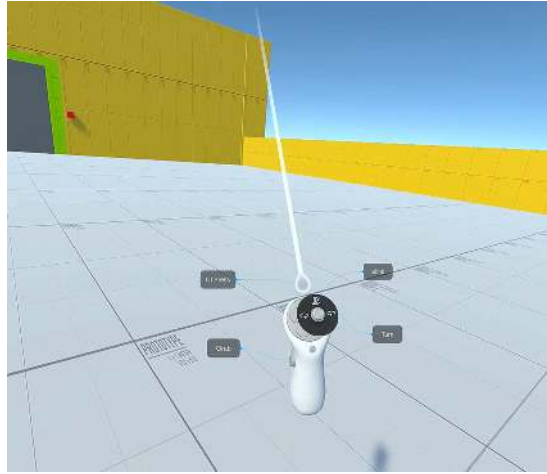


Figure 18: Unity's hand controllers.

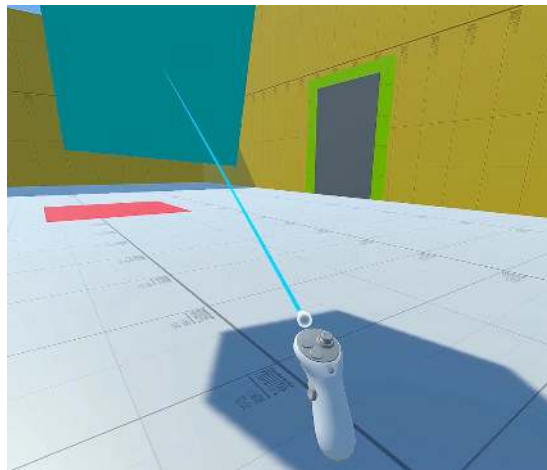


Figure 19: Grabbing an object in Unity.

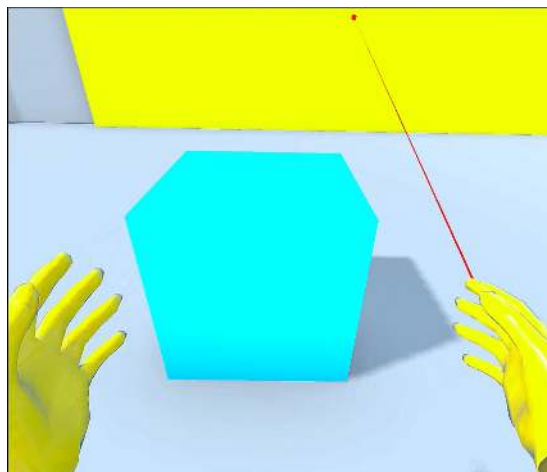


Figure 20: Godot's hand controllers.

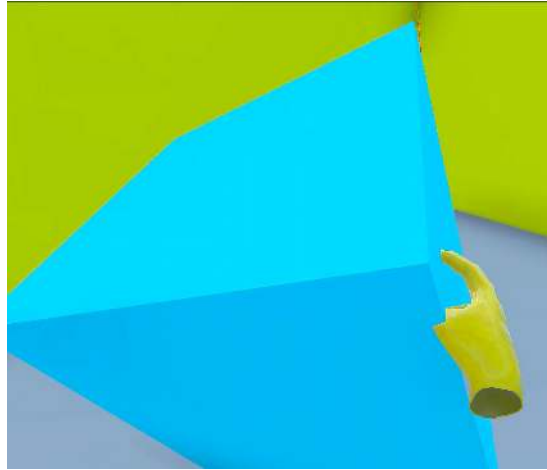


Figure 21: Grabbing an object in Godot.

## Godot

Godot, while versatile in many areas, falls short in the VR department. Although it offers several libraries for out-of-the-box VR functionality, these tools are less mature and harder to use compared to Unity. Key limitations of Godot's VR support include:

- **Limited Documentation:** The documentation for Godot's VR features is sparse and often outdated. This lack of comprehensive and current documentation makes it challenging for developers to understand and implement VR functionalities effectively.
- **Scarcity of Tutorials:** Unlike Unity, Godot has very few tutorials available for VR development. This scarcity of educational resources further complicates the learning curve for developers new to VR.
- **No VR Simulation:** A significant drawback of Godot is the absence of VR simulation support. Developers must have access to a VR headset to test and develop VR applications, which can be a major barrier for those without the necessary hardware.

## Author's Experience

Given the requirement to develop VR games without a VR headset, Godot's limitations in VR support would have presented a significant challenge. The lack of VR simulation tools places developers without VR hardware at a considerable disadvantage, making the development process more time-consuming and less efficient.

When a VR headset was eventually acquired to begin development, multiple obstacles arose due to the lack of proper documentation and outdated or unhelpful tutorials. Considerable time had to be spent reading and exploring the source code of the Godot XR Tools library, spending hours just to implement something as simple as a button press.

## Comparison

The only drawback encountered during VR development with Unity, in comparison to Godot, was the need for extensive adjustments to the graphics settings. In Godot, the graphics fidelity met expectations from the outset, while Unity required considerable tweaking to achieve the desired quality. This was due to Unity's predetermined optimized settings for VR applications, which downgraded the standard graphical fidelity typically offered by the engine.

Unity is the clear winner in the VR support category. Its extensive VR libraries, comprehensive documentation, abundant tutorials, and the ability to simulate VR headsets without physical hardware make it the superior choice for VR development. For developers, especially those without access to VR headsets, Unity's robust and user-friendly VR tools provide a significant advantage over Godot.

### 3.11 Built-in Features

#### Godot

Godot shines in its array of built-in features, designed to streamline development and minimize the need for external tools or extensive custom scripting. The engine's all-in-one interface combines the 2D/3D scene editor, script editor, and asset library, providing a cohesive and intuitive environment for developers. The built-in features that significantly enhance Godot's usability include:

- **CharacterBody:** Simplifies the creation and management of character movement and physics, allowing for easy setup of player controls and behaviors.
- **VehicleBody and VehicleWheels:** Offers ready-made components for vehicle physics, enabling developers to implement realistic vehicle dynamics without additional plugins.
- **Timer:** Provides a straightforward way to handle timing and scheduling of events, essential for managing game loops, animations, and other time-dependent features.

- **Raycast:** Facilitates collision detection and interaction by casting rays from points in the scene, useful for shooting mechanics, line-of-sight checks, and more.
- **Canvas/UI Items:** A comprehensive suite of tools for creating and managing user interfaces, ensuring that developers can build interactive and visually appealing UI elements.
- **AudioStreamPlayer:** Manages audio playback easily, supporting various audio formats and ensuring that sound integration is seamless.
- **BoneAttachment:** Allows objects to be attached to specific bones within a skeletal structure, crucial for character customization and dynamic object attachment.
- **Object Joints:** Provides a variety of joint types for creating complex mechanical systems, enhancing the realism of interactions and physics-based puzzles.
- **Path and PathFollow (Splines):** Supports path-based movement and animations, allowing objects to follow predefined routes smoothly.
- **Skeleton (Bone Hierarchy):** Manages skeletal structures for animations, essential for character rigging and animation blending.
- **SpringArm:** Automatically adjusts the position of attached objects to avoid collisions, useful for camera systems and object interactions.

These features make Godot feel like a set of "building blocks," where developers can intuitively piece together various components to build their games. This approach significantly reduces the need to develop basic functionality from scratch, making the development process more efficient and user-friendly.

## Unity

Unity also offers a robust set of built-in features, though it often requires more setup and customization compared to Godot. While Unity provides a clean and intuitive UI, many foundational components need to be developed or integrated through external assets and plugins. Key built-in features of Unity include:

- **Prefab System:** Allows developers to create reusable game objects, streamlining the creation and management of complex scenes.

- **Component-Based Architecture:** Encourages modular design, where functionality is added to game objects through components, promoting flexibility and reusability.
- **Unity UI System:** A versatile UI system that supports complex interfaces, animations, and interactions, though it may require additional setup for advanced features.
- **Animation System:** Provides powerful tools for creating and managing animations, including the Animator and Animation Controllers, which support complex animation states and transitions.
- **Physics Engine:** Based on NVIDIA's PhysX, Unity's physics engine handles both 2D and 3D physics calculations efficiently, though custom scripting is often needed for specific behaviors.
- **Rendering Pipelines:** Unity offers both the High Definition Render Pipeline (HDRP) and Universal Render Pipeline (URP), catering to different performance needs and visual fidelity requirements.

### Author's Experience

In the author's experience developing a 3D escape room game with Unity, the deep integration with Unity's libraries proved invaluable. Utilizing C#, the author was able to write serializable dynamic dictionaries that seamlessly integrated with Unity's scene editor (code included in appendix A, sub-section A.2). This customization enabled faster development and allowed for more efficient management of game data directly within the editor. The ability to create and manipulate complex data structures using custom tools in the editor significantly streamlined the development process and boosted productivity.

### Comparison

While both engines provide robust built-in features, Godot's extensive out-of-the-box functionality and integrated tools make it the winner in this category. Its comprehensive set of pre-built components and intuitive, all-in-one interface offer a significant advantage for developers seeking a streamlined and efficient development process. Unity, while highly powerful and customizable, often requires more setup and reliance on external assets to achieve the same level of built-in functionality.

### 3.12 Code-Length Comparison: A Realistic Example

When comparing the code length between Godot and Unity, several differences become apparent due to the inherent design and language specifics of each engine.

#### Godot

In Godot, using GDScript, the provided code spans 212 non-empty lines. GDScript's Python-like syntax and dynamic typing contribute to concise and readable code. This brevity helps developers focus more on game logic and functionality rather than boilerplate code.

Godot offers many built-in features and an integrated development environment that reduces the need for extensive setup and external tools. The deep integration of GDScript with Godot's editor further shortens code length, allowing for quick object referencing and scene manipulation.

#### Unity

In Unity, using C#, the provided code spans 231 non-empty lines. C# is a statically-typed language with a more verbose syntax compared to GDScript. This can lead to longer code, as seen in the comparison, but it also provides benefits in terms of type safety and more structured code.

Unity's extensive libraries and robust API offer a wide range of functionalities, but they often require additional lines of code to implement compared to Godot's more integrated approach. The need to frequently use explicit type definitions and handling more detailed configurations also adds to the code length in Unity.

#### Comparison

To summarize, we can refer to the example provided in Figure 7, which offers a focused comparison within a limited scope. In this specific example it is clear that for the same exact functionality, Godot only needed 26 lines of code while Unity needed 34. While the difference in the lines of code may seem negligent for this specific example, when scaling to the full-sized project it results in a significant difference of approximately 27% more lines of code.

Before we proceed, it is important to distinguish code-length from development time. As can be seen in the following sub-section, lower code-length does not equate to lower development time.

This phenomenon can be observed in the scope of this thesis using a specific example. In the previous sub-section it was mentioned that the author was able to write and integrate a new data-structure in Unity using C# in the form of

serializable dictionaries (see Appendix A, sub-section A.2 for implementation). This implementation added a significant amount of lines in the code of the Unity game but ultimately aided the development process in the ease-of-use department and reduced development time.

To conclude the code-length segment, while both engines provide powerful tools and functionalities, Godot's GDScript allows for more concise code, making it easier to write and manage scripts with fewer lines. Unity's C# offers robustness and type safety but tends to result in lengthier code. For developers prioritizing brevity and ease of use, Godot has an edge in code length. However, for those who prefer a more structured and type-safe environment, Unity remains a strong contender despite the longer code.

In summary, the choice between Godot and Unity in terms of code length comes down to a trade-off between concise scripting and the benefits of a statically-typed language.

### 3.13 Development Time

It is important to note that Unity's game was developed after the Godot's game and shared the same logic. Also, the author has had some prior experience with C# and Unity but none with Godot and GDScript.

#### Godot

The total time logged for the development of the Godot project was approximately 23 hours and 56 minutes.

#### Unity

The total time logged for the development of the Unity project was approximately 14 hours and 32 minutes.

#### Comparison

In conclusion, the development time for the Godot project was nearly double that of the Unity project. This discrepancy can be attributed to the extensive time spent on debugging and refining various systems in Godot, as well as the additional features implemented during the development process. Unity's more stable and mature ecosystem, along with its robust libraries and tools, likely contributed to a more efficient development workflow, resulting in less time spent on achieving similar functionality.

### 3.14 Comparison Summary & Verdict

After thoroughly comparing Godot and Unity across various aspects, several key insights emerge:

#### Licensing

- **Godot:** Completely free and open-source, offering unmatched flexibility.
- **Unity:** Free personal version with professional versions available for a fee, suitable for scalable commercial use.

#### Programming Languages

- **Godot:** GDScript offers ease of learning and deep integration, but is hindered by evolving practices and fragmented documentation.
- **Unity:** C# provides stability, extensive resources, and well-established best practices, making it superior for game development.

#### Platform Support

- **Godot:** Broad platform support with a streamlined export process, though some platforms may require third-party services.
- **Unity:** Extensive platform support, particularly strong in emerging technologies like AR/VR, and superior integration for console platforms.

#### Graphics Capabilities

- **Godot:** Basic graphical elements suitable for simple projects or prototyping.
- **Unity:** Robust tools and advanced rendering pipelines for high-fidelity visuals, supported by comprehensive resources.

#### Asset Store

- **Godot:** Free assets with a growing but limited selection.
- **Unity:** Extensive selection of both free and paid assets, supported by a large and active community.

## Community Support

- **Godot:** Enthusiastic and growing community with impactful contributions, though smaller and less resourced compared to Unity.
- **Unity:** Large, active community providing extensive resources, robust documentation, and numerous third-party tools.

## Ease of Use

- **Godot:** All-in-one interface with a comprehensive set of built-in tools, making it exceptionally intuitive and user-friendly.
- **Unity:** Polished UI but requires additional setup and external assets for full functionality, resulting in a steeper learning curve.

## Performance

- **Godot:** Struggles with performance in complex scenes and higher quality assets.
- **Unity:** Advanced rendering pipelines, efficient asset handling, and optimized physics engine make it superior in performance.

## Editor Customization

- **Godot:** Highly customizable through scripting and a modular interface, but limited by the smaller community.
- **Unity:** Extensive customization using C# and a large library of extensions from the Asset Store, supported by a large community.

## VR Support

- **Godot:** Basic VR support with limited documentation and no VR simulation tools.
- **Unity:** Extensive VR libraries, comprehensive documentation, abundant tutorials, and VR simulation tools make it the superior choice for VR development.

## Built-in Features

- **Godot:** Extensive out-of-the-box functionality and integrated tools offer significant advantages for streamlined development.

- **Unity:** Robust built-in features but often requires additional setup and reliance on external assets.

### Code Length

- **Godot:** GDScript allows for concise code, making it easier to write and manage scripts with fewer lines.
- **Unity:** C# provides robustness and type safety but results in lengthier code.

### Development Time

- **Godot:** Longer development time due to extensive debugging and refining processes.
- **Unity:** Shorter development time attributed to a stable and mature ecosystem, robust libraries, and tools.

### Comparison overview

While both engines have their strengths, Unity emerges as the overall winner due to its superior platform support, advanced graphics capabilities, extensive asset store, robust community support, high performance, and comprehensive VR tools. Godot, with its ease of use, built-in features, and open-source nature, remains an excellent choice for smaller projects, indie developers, and those looking for a free and flexible engine. However, for larger, more complex projects requiring advanced features and extensive resources, Unity is the preferred choice.

## 4 Unity vs Mages SDK

### 4.1 Introduction to the MAGES SDK

MAGES NXT is an SDK built upon the Unity Engine by the OramaVR team. It offers an advanced set of tools and features that enhance game development by integrating out-of-the-box solutions for VR, multiplayer support, and custom actions, among other optimizations [12]. We are mentioning the MAGES SDK, as it and other similar frameworks are typically used on top of modern game engines to facilitate development. Therefore, we will also be comparing MAGES to both Unity and Godot.

#### What does MAGES stand for

**Multiplayer with geometric algebra interpolation** Its custom Multiplayer engine operates atop standard networking protocols. It optimizes data sharing by utilizing our custom geometric algebra interpolation engine transmitting only key transformations across the network, while all intermediate calculations are performed locally. This approach enhances performance and responsiveness.

**Analytics engine** Its novel Analytics engine sets a new standard by capturing several hundred events per second, a substantial leap from the current industry norm. These performance-based assessments are stored locally, maximizing flexibility, and can seamlessly be uploaded to the cloud for centralized analysis.

**Geometric algebra deformable animation, cutting and tearing** At the heart of its virtual human animation and deformation lies a Geometric Algebra vertex transformation engine. This engine offers significantly enhanced and robust control over vertex transformations—encompassing translations, rotations, and dilations—resulting in more efficient and lifelike animations.

**Editor in VR with action prototypes** Its algorithm powers an Educational curriculum editor, enabling visual scripting and rapid prototyping of intricate XR learning simulations. This innovative editor is based on a novel learning scene-graph and storyboard Actions, providing a versatile tool for educators and learners alike.

**Semantically annotated deformable, soft and rigid bodies** Its novel particle system, is suitable for real-time elasticity simulations of human tissues and organs, for highly realistic recreation of XR medical training.

## 4.2 Key Features of Mages NXT

### Out of the Box VR Game Controller

Mages NXT simplifies the setup and configuration of VR controllers, making it easy for developers to integrate VR functionality. This feature ensures that VR interactions are smooth and responsive, reducing the time spent on configuring VR hardware, input settings, movement and interaction logic

### Out of the Box Multiplayer Support

Utilizing a custom multiplayer engine based on geometric algebra interpolation, Mages NXT optimizes network performance. This engine transmits only key transformations over the network, performing intermediate calculations locally to enhance responsiveness and reduce latency, making it ideal for real-time multiplayer interactions

### Fully Functioning Scenegraph

The Scenegraph Editor in Mages NXT provides a robust framework for managing game flow and logic. It allows developers to create, manage, and visualize the structure of their game's scenes, including action nodes and flow nodes, which are essential for defining interactions and game progression

### Enhanced Custom-Made UI in Unity

The MAGES SDK customizes Unity's UI to better suit the specific needs of developers working with the SDK. This tailored UI helps streamline the development process and can be seen in Figure 22

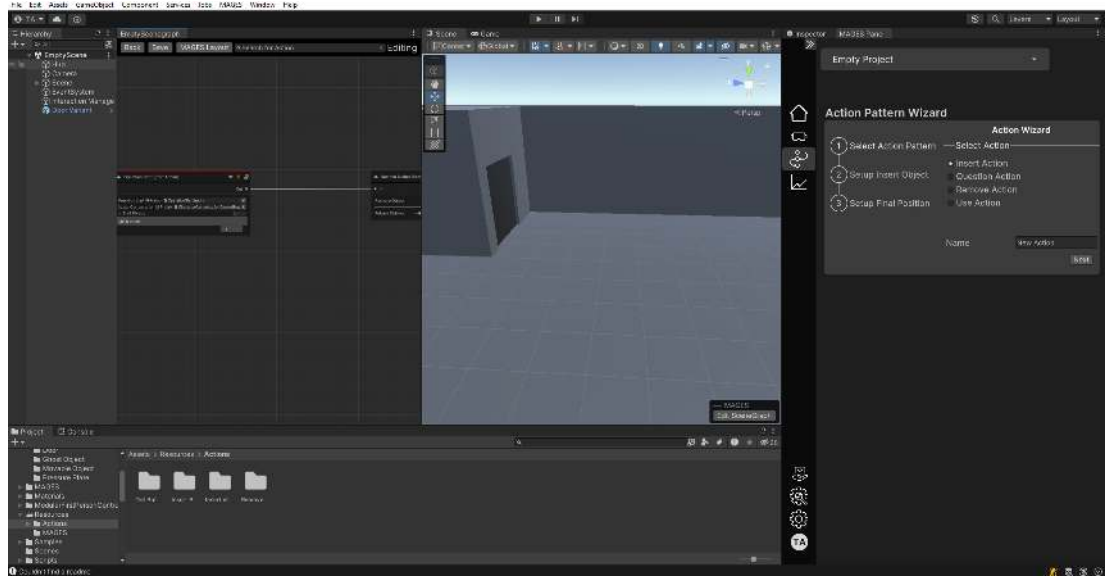


Figure 22: MAGES custom-made UI in Unity.

## Engine Optimizations

Mages NXT incorporates various performance optimizations, ensuring that applications run smoothly even with complex interactions and high-fidelity graphics. These optimizations are crucial for maintaining a seamless user experience in VR and interactive simulations [12]

## No Coding Experience Required

Developers can create fully fleshed-out experiences solely by utilizing the scene-graph, actions, and flows provided by the MAGES NXT SDK, without the need to write a single line of code. This no-code approach significantly lowers the barrier to entry, allowing individuals with varying levels of technical expertise to design and implement sophisticated VR experiences.

## Out of the Box Working User Actions (Action-Patterns)

Mages NXT provides a range of predefined user actions, known as action patterns, which simplify the development of interactive behaviors. These include:

- **Insert Action:** Allows users to place objects correctly, respecting their orientation and position within the virtual space (Figures 23,24).



Figure 23: Insert Action Begin.

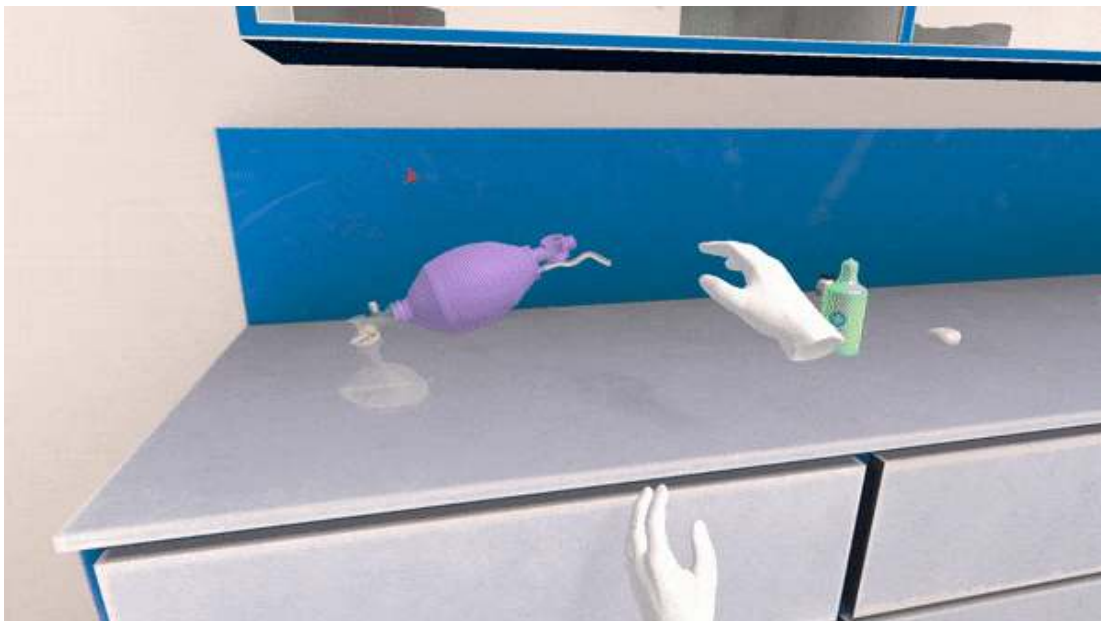


Figure 24: Insert Action Completed.

- **Use Action:** Enables interaction with objects, either over a specified area or with another object (Figure 25).



Figure 25: Use Action Example.

- **Remove Action:** Manages the removal of objects from their designated positions (Figures 26, 27).

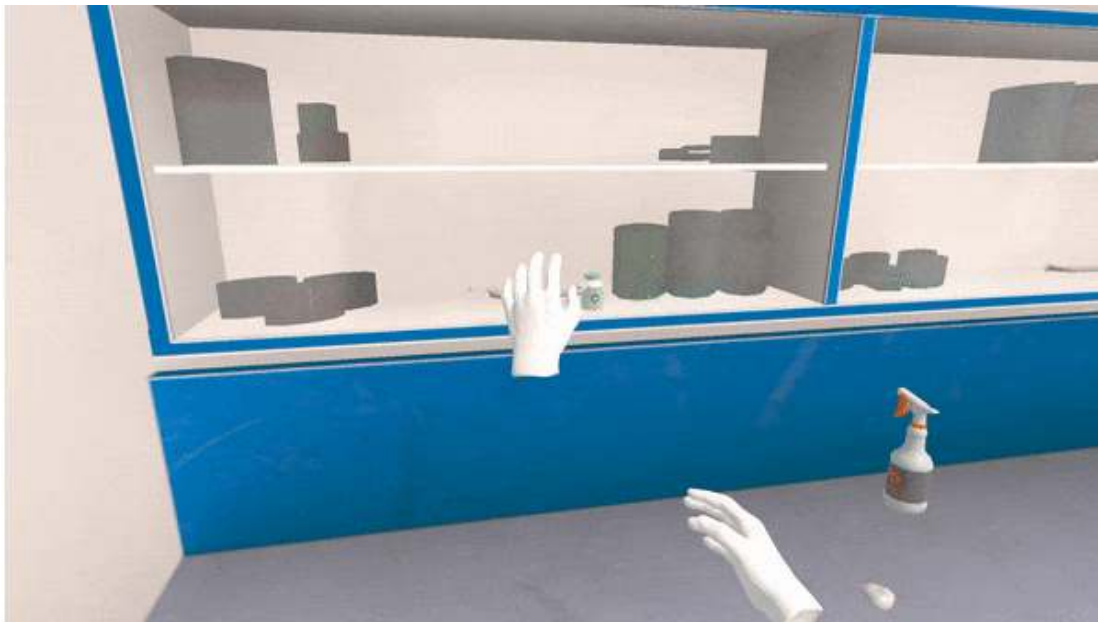


Figure 26: Remove Action Begin.

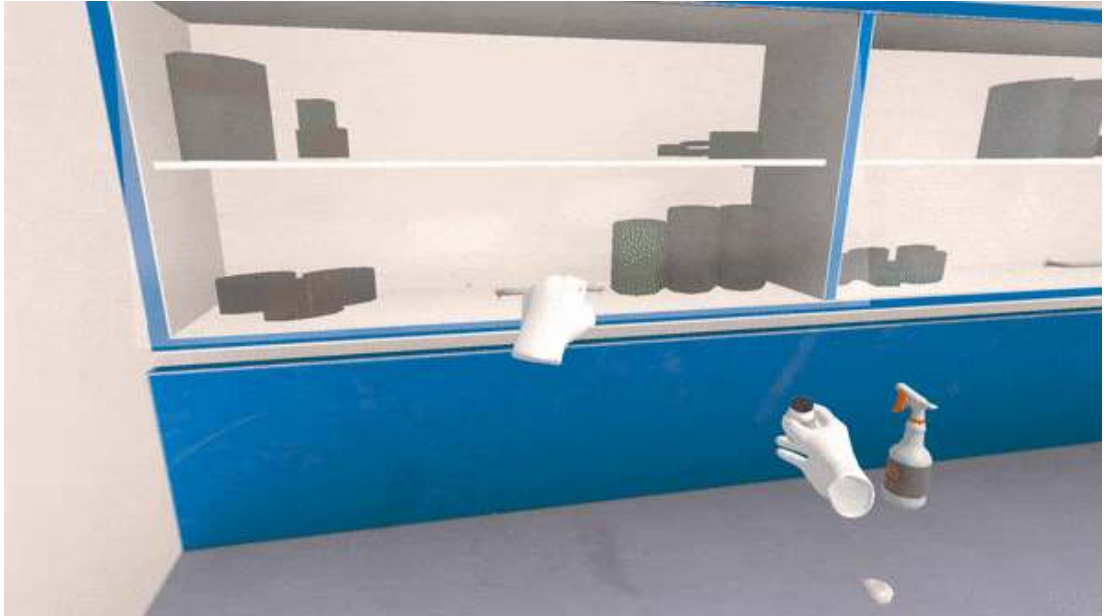


Figure 27: Remove Action Complete.

- **Trajectory Action:** Guides the movement of objects or tools along a predefined path (Figure 28).

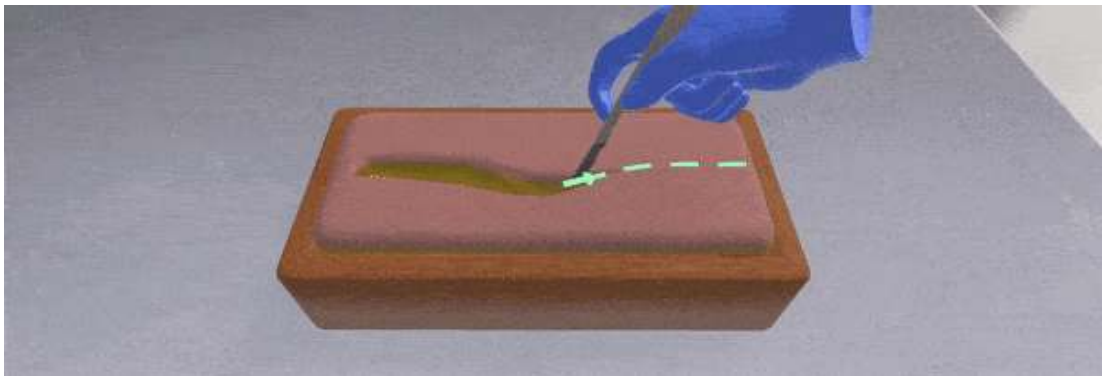


Figure 28: Trajectory Action Example.

- **Activate Action:** Supports actions involving the activation of tools or instruments, such as pressing a trigger on a controller (Figure 29).



Figure 29: Activate Action Example.

- **Question Action:** Facilitates the creation of quizzes or question prompts, allowing users to select single or multiple answers (Figure 30).



Figure 30: Question Action Example.

- **Choice Action:** Lets users choose from multiple options, guiding them through different paths based on their selections.
- **Custom Action:** Developers can create bespoke actions tailored to specific requirements, extending the flexibility of the SDK.

### 4.3 Comparison

By leveraging these features along with all of Unity's strengths and the fact that no coding capabilities are needed, MAGES NXT empowers developers to create rich, interactive experiences with reduced development time and complexity. This SDK is particularly beneficial for applications in medical training, education, and immersive gaming, providing a comprehensive toolkit to bring VR projects to life effectively. Collectively, these attributes position Mages NXT as a superior solution, elevating it above the standard offerings of Unity.

## 5 Godot vs Mages NXT

### 5.1 Introduction

In this section, we will conduct a comparative analysis of Godot and MAGES NXT, with a particular emphasis on VR development due to the VR-centric nature of MAGES NXT. As MAGES NXT is built upon the Unity engine, the core elements of the two engines have already been compared in Section 3. This comparison will therefore highlight the specialized VR capabilities and features that distinguish MAGES NXT.

### 5.2 Out-of-the-Box Functionality

#### Godot

Godot provides a robust set of built-in features, which include tools for character movement, physics, and UI creation. While these features are powerful, they often require significant customization and coding to fully utilize in VR projects. Developers must piece together various components and create custom scripts to achieve complex interactions and immersive VR experiences.

#### MAGES NXT

MAGES NXT excels in out-of-the-box functionality, particularly for VR development. The SDK includes a comprehensive set of pre-configured tools and templates that allow developers to quickly set up VR controllers, interaction patterns, and multiplayer environments. Key features such as scenegraphs, action nodes, and flow nodes are designed to work seamlessly without the need for extensive coding, significantly reducing development time and complexity. This turnkey approach empowers developers to focus on the creative aspects of their projects rather than the technical intricacies.

### 5.3 Documentation and Community Support

#### Godot

One of the main challenges with Godot is its fragmented and often outdated documentation. While the community is active and growing, the available resources can be inconsistent, making it difficult for developers to find accurate and up-to-date information. This lack of comprehensive documentation can be a significant hurdle, especially for those new to the engine or attempting complex VR projects.

## **MAGES NXT**

In contrast, MAGES NXT provides thorough and well-organized documentation that covers all aspects of the SDK. The documentation includes detailed guides, tutorials, and examples that make it easy for developers to get started and resolve any issues they encounter. Additionally, the support infrastructure includes active forums and community channels where developers can seek help and share insights. This comprehensive support system ensures that developers have the resources they need to efficiently navigate the development process.

### **5.4 Coding Requirements**

#### **Godot**

Godot uses GDScript, a Python-like language, which, while easy to learn, still requires developers to write significant amounts of code for implementing VR functionality. Custom scripts are often necessary to handle interactions, physics, and other complex behaviors, which can increase the development time and effort required.

#### **MAGES NXT**

One of the standout features of MAGES NXT is its no-code approach to VR development. Developers can create fully functional VR experiences using the scenegraph, actions, and flows provided by the SDK without writing a single line of code. This approach lowers the barrier to entry, making advanced VR development accessible to individuals with varying levels of technical expertise. By simplifying the development process, MAGES NXT enables faster prototyping and iteration, allowing developers to bring their VR projects to life more efficiently.

### **5.5 Graphics Capabilities**

#### **Godot**

Godot offers functional graphics capabilities, including support for 2D and 3D rendering, materials, and shaders. However, achieving high-fidelity graphics often requires significant manual adjustments and the creation of custom shaders. The engine's default lighting and shadowing systems are relatively basic, which can limit the visual quality of more complex projects.

## **MAGES NXT**

MAGES NXT leverages Unity’s advanced graphics capabilities, providing developers with access to high-quality materials, sophisticated rendering pipelines, and realistic lighting and shadowing systems. The SDK enhances these features with additional tools for creating lifelike animations and immersive environments, making it easier to achieve high visual fidelity in VR projects. The built-in optimizations ensure smooth performance even with complex scenes and interactions.

## **5.6 Multiplayer Support**

### **Godot**

While Godot supports multiplayer functionality, it often requires extensive coding and the integration of third-party tools to achieve robust multiplayer experiences. Setting up networking, synchronization, and other multiplayer features can be complex and time-consuming, especially for VR projects.

### **MAGES NXT**

MAGES NXT includes out-of-the-box multiplayer support with a custom engine based on geometric algebra interpolation. This engine optimizes network performance by transmitting only key transformations and performing intermediate calculations locally, resulting in enhanced responsiveness and reduced latency. The SDK simplifies the setup of multiplayer environments, allowing developers to focus on creating engaging multiplayer interactions without the need for complex networking code.

## **5.7 Comparison**

In conclusion, MAGES NXT offers a superior solution for VR development compared to Godot, particularly due to its out-of-the-box functionality, comprehensive documentation, no-code development approach, advanced graphics capabilities, and robust multiplayer support. While Godot is a powerful and flexible engine, the specialized features and optimizations provided by MAGES NXT make it a more efficient and accessible choice for developers aiming to create rich, interactive VR experiences. By leveraging these strengths, MAGES NXT positions itself as an ideal tool for both novice and experienced developers seeking to bring their VR projects to life with reduced development time and complexity.

## 6 Comparison Summary & Verdict

### 6.1 Comparison Summary

Table 1 contains a condensed summary for each metric of each engine examined in the scope of the thesis.

| Feature               | Godot                                                                                                                                                                                    | Unity                                                                                                                                                                                                                    | MAGES                                                                                                                                                                         |
|-----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Licensing             | Free and Open Source (MIT License). Godot is completely free to use with no strings attached, offering full access to the source code and allowing for modifications and redistribution. | Free personal version available with revenue caps; professional versions available for a fee, allowing commercial use with access to more advanced features and services.                                                | Free. MAGES SDK from OramaVR is offered without cost, making it an attractive option for developers focusing on VR applications without incurring additional expenses.        |
| Programming Languages | Supports GDScript (Python-like), C#, VisualScript, and C++. GDScript is designed for ease of use, while C++ offers high performance for critical sections.                               | Primarily uses C#, a powerful and widely-used programming language known for its strong typing and extensive libraries, facilitating complex game development.                                                           | C# optional. Provides flexibility by allowing developers to use C# or other supported languages, catering to different coding preferences and expertise levels.               |
| Platform Support      | Wide range, including Windows, macOS, Linux, HTML5, Android, iOS, and more. This broad platform support makes Godot versatile for both desktop and mobile development.                   | Extensive platform support, including Windows, macOS, Linux, HTML5, Android, iOS, and game consoles. Unity's robust platform support is a major reason for its popularity among developers targeting multiple platforms. | Desktop VR, Android, iOS. Primarily focused on VR development, MAGES SDK supports key VR platforms along with mobile, allowing developers to create immersive VR experiences. |

| Feature                | Godot                                                                                                                                                                                   | Unity                                                                                                                                                                                                      | MAGES                                                                                                                                                                         |
|------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Graphical Capabilities | Supports 2D and 3D with advanced features, though generally considered less powerful for high-end 3D applications. It excels in 2D game development with a strong, easy-to-use toolkit. | Robust support for both 2D and 3D, known for high-end 3D capabilities. Unity's graphical engine is highly optimized for complex 3D rendering, making it suitable for AAA game development.                 | Same as Unity. Offers comparable graphical capabilities, leveraging Unity's strengths to provide high-quality 2D and 3D graphics, particularly optimized for VR environments. |
| Asset Store            | Growing selection of assets and plugins, generally less expensive than Unity's offerings. The Godot Asset Library provides a variety of community-contributed assets and tools.         | Extensive asset store with a wide range of assets and tools, including models, scripts, and complete project templates. Unity Asset Store is renowned for its comprehensive and high-quality resources.    | MAGES SDK leverages Unity's extensive asset store, giving developers access to a wide array of assets and plugins tailored for VR development.                                |
| Community Support      | Active and growing community with a strong culture of open-source contributions. The community is vibrant and frequently contributes to the engine's development and support.           | Very large and active community with extensive professional support available. Unity's community is one of the largest in the game development world, providing abundant resources, tutorials, and forums. | Almost none. The community for MAGES SDK is still developing, offering limited support compared to more established engines, which can be challenging for new developers.     |

| Feature              | Godot                                                                                                                                                                                         | Unity                                                                                                                                                                                                                 | MAGES                                                                                                                                                                                      |
|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Ease of use          | Often considered easier for beginners, especially for 2D game development. Godot's user-friendly interface and scripting language make it accessible for newcomers.                           | Steeper learning curve due to its extensive feature set, but offers comprehensive documentation and tutorials. Unity's complexity can be daunting, but its detailed resources help ease the learning process.         | Easy to develop apps that use step-by-step actions. MAGES SDK simplifies VR development with intuitive tools and straightforward workflows, making it accessible even for those new to VR. |
| Performance          | Efficient, especially for 2D and smaller 3D games. Performance depends on GDNative for heavy computations, allowing developers to write optimized code in C++ for performance-critical tasks. | High performance, capable of handling large and complex 3D games effectively. Unity's engine is optimized for high-end performance, suitable for developing demanding games and applications.                         | High performance optimized for VR. MAGES SDK is designed to deliver excellent performance in VR environments, ensuring smooth and immersive experiences on supported devices.              |
| Editor Customization | Highly customizable editor through scripting. Developers can tailor the Godot editor to their needs using GDScript, enhancing their workflow and productivity.                                | Customizable to a certain extent, with more options available through plugins. Unity allows for considerable editor customization, though it often relies on third-party plugins to achieve specific functionalities. | Same as Unity. Leverages Unity's customization options, allowing developers to modify the editor to fit their specific VR development requirements.                                        |

| Feature           | Godot                                                                                                                                                   | Unity                                                                                                                                                                             | MAGES                                                                                                                                                                                       |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| VR Support        | Basic VR support, growing with community contributions. While still maturing, Godot's VR capabilities are expanding thanks to active community efforts. | Strong VR support with built-in tools and assets. Unity provides comprehensive VR development tools, making it a popular choice for creating VR experiences.                      | Mainly supports VR with built-in tools and assets. MAGES SDK is specialized for VR, offering a streamlined set of tools and assets specifically designed to facilitate VR development.      |
| Built-in features | Offers a wide variety of built-in features that make development easier to start, including a visual editor, animation tools, and a robust node system. | Starting Unity projects do not offer the same kind of out-of-the-box functionality as Godot, but there are many free plug-ins to take their place, enhancing Unity's flexibility. | Assets, multiple actions, scenograph. MAGES SDK provides built-in assets and tools designed to simplify VR development, focusing on quick setup and ease of use.                            |
| Code length       | GScript is a Python-like language, which offers functionality with fewer lines of code, making it ideal for rapid development and prototyping.          | Unity uses C#, a statically-typed language that may require more lines of code but provides robustness and performance, essential for larger projects.                            | Optional, slim to none. MAGES SDK offers flexibility in coding, allowing developers to choose their preferred language and coding style, potentially minimizing code length and complexity. |

Table 1: Summary Comparison Table

## 6.2 Verdict

The thesis concludes that, while all three engines have their distinct strengths, **Unity emerges as the overall winner**. Its advanced graphics capabilities, extensive platform support, and comprehensive asset store make it the go-to choice for developers working on large-scale and complex projects. Unity also shines in VR development, offering robust tools and strong community support, which are crucial for both novice and experienced developers. This versatility and depth of resources position Unity as the best option for a wide range of applications, from high-fidelity games to immersive VR experiences.

**Godot**, on the other hand, is an excellent choice for smaller projects, particularly in 2D game development. Its free and open-source nature makes it accessible and appealing to indie developers and hobbyists. However, it falls short in high-end 3D graphics and VR support, making it less suitable for more demanding projects. Godot's ease of use and strong built-in features are significant advantages, but the engine still has room to grow, especially in terms of community resources and documentation.

**MAGES NXT** stands out as a specialized tool for VR development, particularly in the fields of education and medical simulations. Built on Unity, it offers a no-code development approach, making it accessible to those with limited coding experience. MAGES NXT excels in providing out-of-the-box solutions for VR and multiplayer support, making it an ideal choice for projects that require immersive and interactive environments.

In summary, Unity is the most versatile and resource-rich engine, suitable for a wide range of projects. Godot is ideal for smaller, less graphically intensive projects, particularly in 2D, while MAGES NXT is the preferred choice for specialized VR applications, especially in educational and medical simulations. The choice of engine should be guided by the specific requirements and goals of the project at hand.

## References

- [1] 6Sense. Unity - market share, 2024. URL: <https://6sense.com/tech/game-development/unity-market-share>.
- [2] Game Developer. The perfect organism: The ai of alien: Isolation, 2017. URL: <https://www.gamedeveloper.com/design/the-perfect-organism-the-ai-of-alien-isolation>.
- [3] Godot Docs. Customizing the interface, 2024. URL: [https://docs.godotengine.org/en/stable/tutorials/editor/customizing\\_editor.html](https://docs.godotengine.org/en/stable/tutorials/editor/customizing_editor.html).
- [4] Unity Documentation. Built-in 3d physics, 2024. URL: <https://docs.unity3d.com/Manual/PhysicsOverview.html>.
- [5] Unity Documentation. Create custom editors with imgui, 2024. URL: <https://docs.unity3d.com/Manual/editor-CustomEditors.html>.
- [6] Cry Engine. Cryengine | the complete solution for next generation game development, 2024. URL: <https://www.cryengine.com/>.
- [7] Gotcha Gotcha Games Enterbrain. Make your own game with rpg maker, 2024. URL: <https://www.rpgmakerweb.com/>.
- [8] Epic Games. Unreal engine: The most powerful real-time 3d creation tool, 2024. URL: <https://www.unrealengine.com/en-US>.
- [9] Godot. Godot asset library, 2024. URL: <https://godotengine.org/asset-library/asset>.
- [10] Godot. Godot engine - free and open source 2d and 3d game engine, 2024. URL: <https://godotengine.org/>.
- [11] Game Maker. Make 2d games with gamemaker | free video game maker, 2024. URL: <https://gamemaker.io/en>.
- [12] ORamaVR. MAGES NXT documentation, 2024. URL: <https://mages-docs.oramavr.com/en/latest/>.
- [13] Reddit. MAGES SDK : MAGES NXT is the next step in XR development, 2024. URL: [https://www.reddit.com/r/UnityAssets/comments/1d1rgzo/mages\\_sdk\\_mages\\_nxt\\_is\\_the\\_next\\_step\\_in\\_xr/](https://www.reddit.com/r/UnityAssets/comments/1d1rgzo/mages_sdk_mages_nxt_is_the_next_step_in_xr/).

- 
- [14] Reddit. Why you are making 2d instead of 3d?, 2024. URL: [https://www.reddit.com/r/Unity2D/comments/wotoss/why\\_you\\_are\\_making\\_2d\\_instead\\_of\\_3d/](https://www.reddit.com/r/Unity2D/comments/wotoss/why_you_are_making_2d_instead_of_3d/).
  - [15] Unity Technologies. Unity real-time development platform | 3d, 2d, vr ar engine, 2024. URL: <https://unity.com/>.
  - [16] Alan J. W. Thorn. 3d lighting and materials. pages 161–200, 2020. doi: 10.1007/978-1-4842-5908-5\_5.
  - [17] Unity. Unity asset store, 2024. URL: <https://assetstore.unity.com/>.
  - [18] Unity. Unity documentation, 2024. URL: <https://docs.unity.com/>.

## A Appendix

### A.1 Godot development impressions

Godot is an open-source, lightweight and intuitive game engine that has managed to simplify and streamline many aspects of game development.

In this section, the author will break down the major unique features of Godot that make it stand out against its competition and made a great first impression when developing the game.

#### UI:

Godot offers an all-in-one editor, where the scene editor, code editor, assets, hierarchy and scene inspector all exist within the same window.

- The scene editor offers most of the functionality one would expect from a 3D game engine and it works great for the most part.
- The node based hierarchy is a very powerful system that forces users to comply with the rules of Godot, making the development process less volatile.
- Scene inspector is almost identical to the object inspector in the Unity Engine
- Assets are displayed in a list format just like a file explorer's UI. It is clean and works as expected.
- The code editor is deeply integrated within the engine, offering easy node and object referencing functionality, which is critical to the process of developing a game in Godot, using mouse drag and drop.

| Pros                                                                                                                    | Cons                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Seamless transition between coding and editing the scene                                                                | In some cases, namely the scene inspector, the UI can feel clustered causing users to miss features which are hidden in an endless loop of drop-down menus which are hard to see.                                                                                        |
| The UI is super responsive due to the lightweight nature of the engine and workflow is subsequently faster              | The code editor, while deeply integrated, breaks after using editor add-ons like VIM. E.g. auto-complete and class member/function previews break completely.                                                                                                            |
| Users rarely feel lost when developing because everything they would ever use is always accessible from the same window | Using an external editor becomes an extreme disadvantage when taking into consideration the integration of the native editor and slows down workflow and also removes many quality of life features Godot offers.                                                        |
|                                                                                                                         | In my experience, using external editors break the saving of files because Godot opens two versions of each file, one in the native editor and one in the external. This causes an endless loop of saving and loading which results in the game not running as intended. |

**GDScript:**

Godot's powerful scripting language. Bears close resemblance to python and Kotlin, making it easy to start writing well structured code.

| Pros                                                                                                                                            | Cons                                                                                                                                                                                                                                                                                                                                                                         |
|-------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Offers both dynamic and static variable typing support.                                                                                         | One major downside is the confusing and outdated documentation available for the language. To elaborate, due to Godot being in early development, there have been a lot of changes through the updates for GDScript. Much of the documentation and forums online have outdated code and solutions to problems, making it very difficult to find reliable information online. |
| Introduces signals, a convenient system for objects communicating with each other or for coding tasks that require an async condition to occur. | It is early in development and is missing some useful and widely used features and data structures (e.g., Dynamic maps/dictionaries).                                                                                                                                                                                                                                        |
| Syntax is very similar to Python, making it very easy to learn and write clean and quick code.                                                  | Missing syntactic sugar that makes other languages easier and more "crafty" to use. Missing list comprehensions, null check, and ternary operators, etc.                                                                                                                                                                                                                     |
| It is deeply integrated with Godot, so it offers great drag-and-drop referencing of objects in the editor.                                      | Due to the continuously updating nature of Godot, it is missing the edge to the tried and tested practices and knowledge of Unity's libraries and C#.                                                                                                                                                                                                                        |

**Node System:**

Godot is based around the Node system. Every object in Godot is a Scene and the root of all the scenes is a Node. The highest level - most abstract Node is "Node" and every other node is derived from it as seen in figure 31.

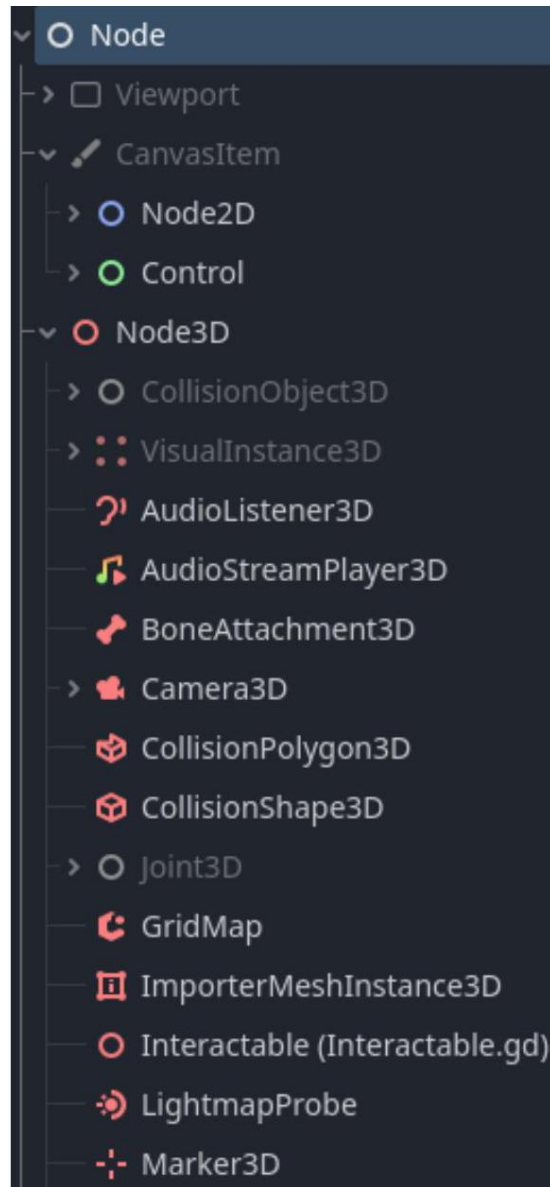


Figure 31: Node List.

In Figure 32 we can observe a common scene hierarchy in Godot. In practice, this means that each Node extends its parent and consequently extends all the parent Nodes above it.

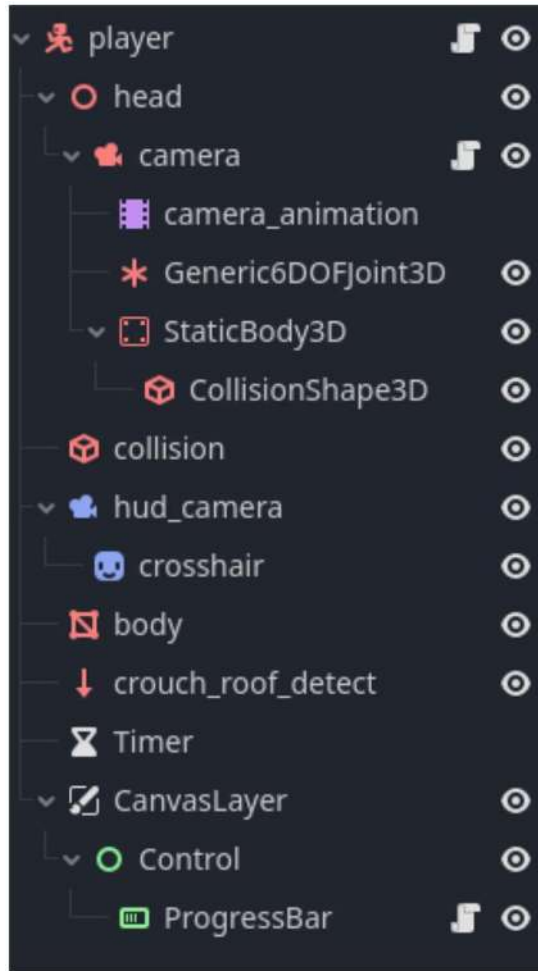


Figure 32: Example Scene Hierarchy.

Creating custom classes, especially abstract ones, can be challenging because all objects inheriting from the abstract class must also inherit from the same parent node that the abstract class extends. To clarify with an example: Suppose we have an abstract class, as shown in Figure 30, that extends `Area3D`. This abstract class can only be extended by nodes that also inherit from `Area3D`, and no other node types.

```

extends Area3D

class_name TestClass
  
```

Figure 33: GDScript Abstract class.

| Pros                                                 | Cons                                                          |
|------------------------------------------------------|---------------------------------------------------------------|
| Intuitive, clean and mostly fast development process | Steep learning curve at the beginning of the learning process |

### Animation:

Godot's animation system is as intuitive as it gets, letting the developer use any available object state as a key frame, with a User Interface as clean and easier to use than Unity's (Figures 34, 35).

When the key symbols are pressed a key frame is created.

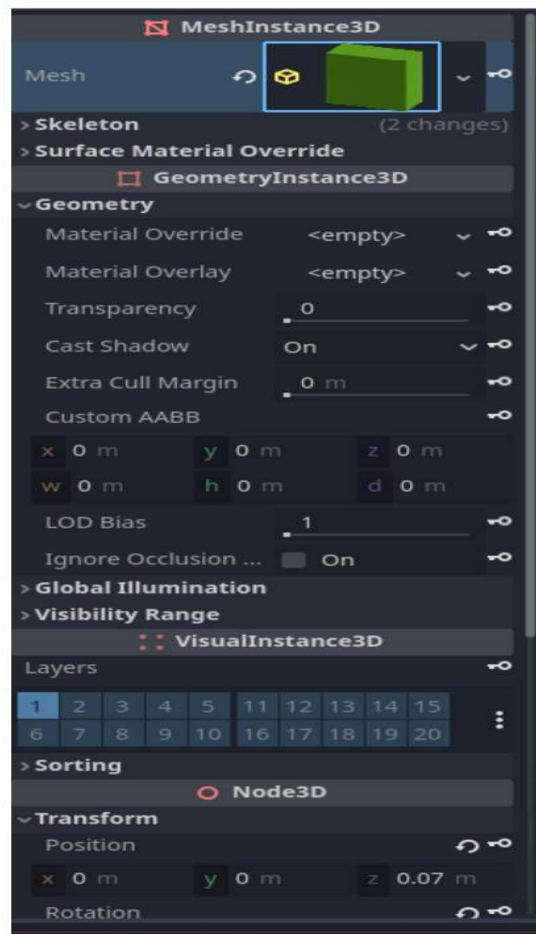


Figure 34: Godot's key frame creation system.

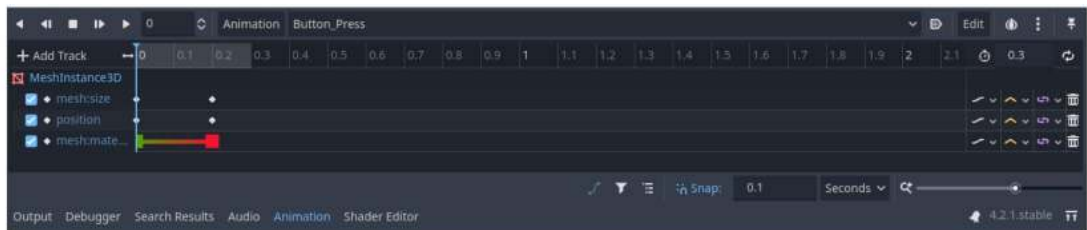


Figure 35: Animation Player's UI

For the button press animation, the author uses the meshes' size, position and material as key frame transitions.

Determining the key frame's state is as simple as moving and changing the object's attributes manually in the scene and creating the key frames as mentioned above.

## A.2 Serializable Dictionary in C#

This is the serializable dictionary implementation mentioned in Section 3, subsection 3.11. The code implementation can be seen in the Figure 36.

```

1  using System;
2  using System.Collections.Generic;
3  using System.Linq;
4  using UnityEngine;
5  using UnityEngine.Serialization;
6
7  [Serializable]
8  public class SerializableKeyValuePair<TKey, TValue> {
9      public TKey key;
10     public TValue value;
11
12     public SerializableKeyValuePair(TKey key, TValue value) {
13         this.key = key;
14         this.value = value;
15     }
16 }
17
18 [Serializable]
19 public class SerializableDictionary<TKey, TValue> : ISerializationCallbackReceiver {
20     [SerializeField] private List<SerializableKeyValuePair<TKey, TValue>> list = new List<SerializableKeyValuePair<TKey, TValue>>();
21
22     public void Add(TKey key, TValue value) {
23         list.Add(new SerializableKeyValuePair<TKey, TValue>(key, value));
24     }
25
26     public bool TryGetValue(TKey key, out TValue value) {
27         var pair = list.FirstOrDefault(p => EqualityComparer<TKey>.Default.Equals(p.key, key));
28         if (pair != null) {
29             value = pair.value;
30             return true;
31         }
32
33         value = default;
34         return false;
35     }
36
37     public List<TKey> Keys() {
38         return list.Select(pair => pair.key).ToList();
39     }
40
41     public List<TValue> Values() {
42         return list.Select(pair => pair.value).ToList();
43     }
44
45     public List<SerializableKeyValuePair<TKey, TValue>> Pairs() {
46         return list;
47     }
48
49     public void OnBeforeSerialize() {
50     }
51
52     public void OnAfterDeserialize() {
53     }
54
55     public List<SerializableKeyValuePair<TKey, TValue>>.Enumerator GetEnumerator() {
56         return list.GetEnumerator();
57     }
58
59     public List<SerializableKeyValuePair<TKey, TValue>> GetList() {
60         return list;
61     }
62 }

```

Figure 36: Unity Serializable Dictionary

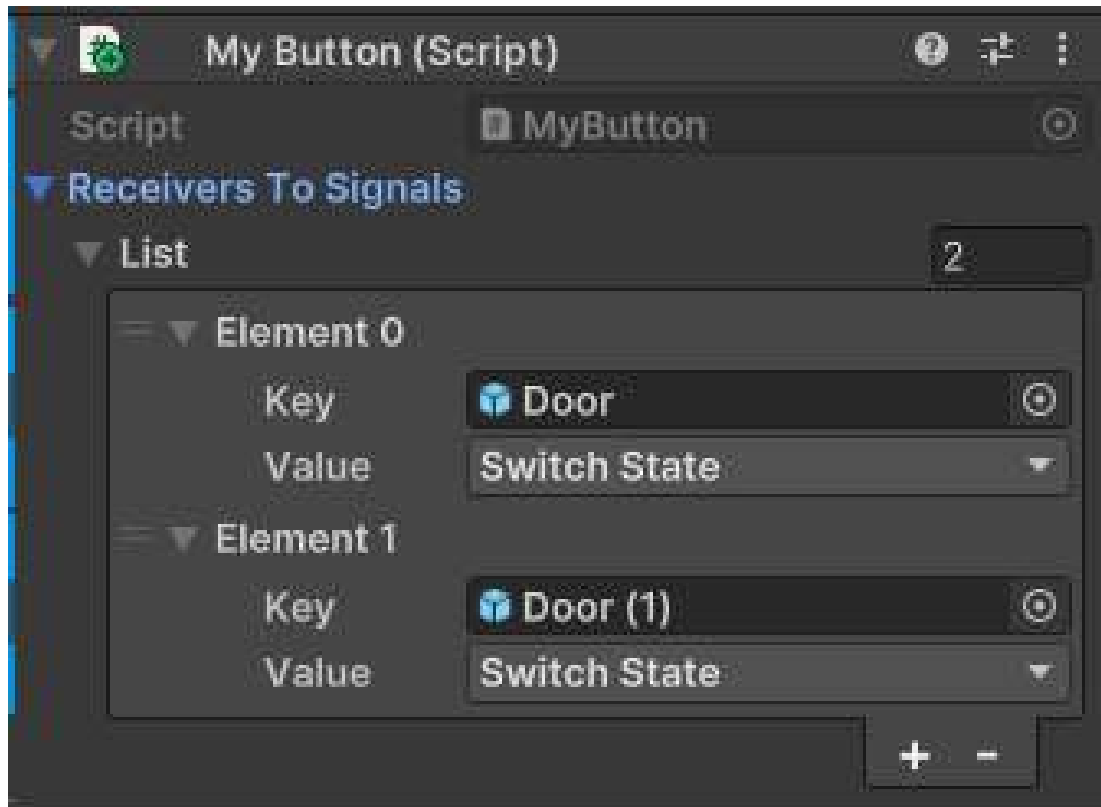


Figure 37: Serializable Dictionary in-editor result

This implementation enables the user to set a list of elements of type "Dictionary" within Unity's scene editor, allowing direct reference to scene objects, as illustrated in Figure 37. This feature significantly reduces development time by offering a quick and organized method for developers to specify which objects are influenced by the "Button" object and how they are affected.